A Novel Application of Ant Colony Optimization to k-defective Maximum Edge Bicliques

Kiyaan Pillai

***Abstract*—The maximum k-defective edge biclique (k-MDB) problem is an emerging NP-hard optimization. We adapt Ant Colony Optimization (ACO) to the k-MDB problem, introducing a local neighborhood search to improve solution quality. In many instances, ACO produces bicliques with 80% or more edges than previously proposed heuristics. Additionally, ACO provides a tighter lower bound on the size of the k-MDB and can cut brute force search time by more than half.**

I. Introduction

A bipartite graph is a common graph structure where one set of vertices is connected solely to another. Bipartite graphs can model user-product behavior, person-place interaction, and more. A biclique is a set of vertices on a bipartite graph such that each vertex on one side is connected to each vertex on the other. The maximum edge biclique problem aims to identify the biclique with the most edges on a bipartite graph. The problem has applications in online recommendation systems, fraud detection, and community detection [1].

One example of bicliques' utility is in e-commerce fraud detection, where a bipartite graph could represent users and the products they buy. A set of accounts that purchase overlapping sets of items may represent a coordinated fraud ring, yielding a subgraph that is almost fully connected across the two sides. Requiring a perfect biclique would discard this signal whenever a few transactions are omitted deliberately or lost to incomplete logging. A dense, near-biclique structure remains interpretable under small amounts of noise; perfect completeness does not matter.

Generally, real-world data often introduces noise to graphs, and structures with relatively few missing connections may still be relevant. A biclique that is allowed up to k missing edges, or a k-defective biclique (k-DB), can overcome this real-world noise. Identifying the maximum k-defective edge biclique (k-MDB) remains an emerging field.

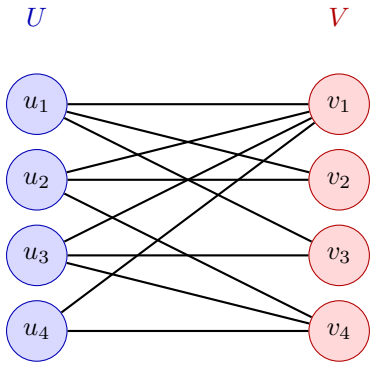


Fig. 1. Example bipartite graph $G = (U, V, E)$

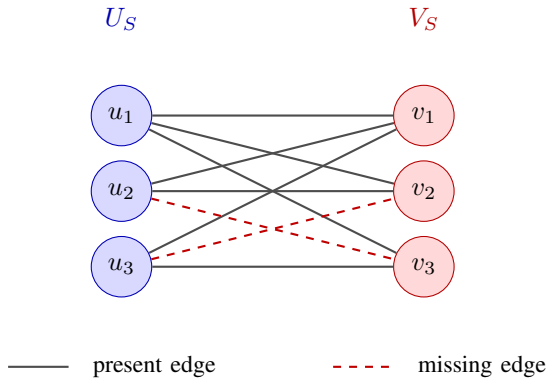


Fig. 2. A 2-defective biclique present in Figure 1. Every vertex in U_S is connected to every vertex in V_S except for two missing edges. Since $|\bar{E}(S)| = 2$, this is a 2-defective biclique. In fact, it is a k-defective biclique for any $k \geq 2$.

Maximizing edge count alone is not enough for real-world interpretability. Without a lower bound on each side's size, the optimum is often a lopsided subgraph with many edges but little practical significance. In the case of fraud, a single popular item purchased by thousands of unrelated users could be a massive biclique with one vertex opposite many others, but it signals mass-market demand rather than a coordinated ring. We therefore seek only θ-feasible k-DB: k-DB where both sides have at least θ vertices, where θ is a user-defined parameter.

The k-MDB problem is NP-hard. Unless P = NP, there is no algorithm that can run in polynomial time and is guaranteed to find the maximum solution. When the absolute optimum is not necessary, approximation techniques are useful to significantly reduce computation while providing meaningfully large bicliques. The only heuristic in literature for this problem is proposed by Cui et al., referred to in this paper as the θ-heuristic. Cui et al. also propose a brute-force algorithm to find the k-MDB called branch-and-pivot.

Dorigo et al. [2] proposed a metaheuristic known as Ant Colony Optimization (ACO) for exploring graphs. Initially used on finding the shortest path in the Traveling Salesman Problem, ACO has been adapted to many graph problems, such as vehicle routing [5], quadratic assignment [6], graph coloring [7], and the maximum clique problem [8].

We introduce an ACO-inspired heuristic for the k-defective maximum edge biclique (k-MDB) problem, incorporating a neighbor-scope restriction that reduces the set of vertices considered during construction. Across 37 benchmark graphs, ACO frequently finds larger and more often θ-feasible solutions than the existing θ-heuristic, although at substantially higher computational cost. We further show that these stronger solutions can improve branch-and-pivot search, reducing its wall-clock time by up to 56% on tested instances. Together, these results demonstrate ACO as a complementary approach to existing k-MDB heuristics, trading increased search cost for improved solution quality.

II. Background

A. Definitions

Define a bipartite graph as a graph $G = (U, V, E)$, where U and V are two disjoint sets of vertices, and $E \subseteq U \times V$ is the set of edges between U and V. Let $S = (U_S, V_S)$ denote a set of vertices where $U_S \subseteq U$ and $V_S \subseteq V$. Let $E_S = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \in E\}$ be the set of edges in a subgraph S. Define $\bar{E}(S) = \{(u, v) \mid u \in U_S \wedge v \in V_S \wedge (u, v) \notin E\}$ be the set of non-edges in a subgraph. Let a k-defective biclique be a subgraph S where $|\bar{E}(S)| \leq k$. Because every possible edge runs between the two sides of a biclique, a subgraph with $|U_S|$ and $|V_S|$ vertices has at most $|U_S||V_S|$ edges. For a k-defective biclique, $|E(S)| = |U_S||V_S| - |\bar{E}(S)|$, with $|\bar{E}(S)| \leq k$.

Define the neighbors of a vertex u in a subgraph S as $N_S(u) = \{v \in S \mid \{u, v\} \in E(S)\}$. Define the nonneighbors of a vertex as $\bar{N}_S(u) = \{v \in S \mid (u, v) \notin E(S)\}$. Define the degree of a vertex as $d_S(u) = |N_S(u)|$, and the nondegree as $\bar{d}_S(u) = |\bar{N}_S(u)|$.

Define the maximum k-MDB as the k-DB with the largest number of edges in a graph G. Let θ be a user-defined minimum side size. The algorithms proposed in this paper seek the maximum k-MDB D^* with $|U_{D^*}| \geq \theta$ and $|V_{D^*}| \geq \theta$ as described in Section 1. We require that $\theta > k$. A vertex on one side can have at most $|V_S|$ missing edges. If $|V_S| \leq k$, then even a vertex with zero neighbors can satisfy the k-defective constraint. Therefore requiring $\theta > k$ ensures every vertex in a θ-feasible k-DB must have at least one neighbor.

B. Computational Complexity

Finding the maximum k-MDB on a bipartite graph is an NP-hard optimization problem. The maximum clique problem reduces to the k-MDB problem [1]. Because the maximum clique is NP-hard, the k-MDB problem is NP-hard as well.

C. Previous Work

Similar dense subgraph models provide precedent for the k-MDB. In non-partite graphs, *defective cliques*, cliques that are missing up to a certain number of edges, have been used to recover nearly complete structures from noisy biological interaction graphs [12]. On bipartite graphs, relaxed models include k-biplexes, which permit each vertex up to k nonneighbors in the biplex [15], and k-defective bicliques, which have a tighter restriction of only allowing up to k total missing edges. The k-MDB generalizes to the maximum edge biclique when $k = 0$ [1], [11].

Cui et al. [1] proposed a brute-force approach to the k-MDB problem, as well as an initial heuristic designed to find a k-MDB with exactly θ vertices on one side of the graph, described in Section 1. Concurrently, Wang et al. [11] studied the related problem of finding a k-defective biclique with the largest vertex count rather than edge count. Because their objective differs from ours, we do not benchmark against their algorithm.

Ant Colony Optimization is a natural metaheuristic for problems where high-quality solutions are built by iteratively adding vertices to a subgraph. In the maximum clique problem, ACO algorithms construct cliques greedily while using pheromone to reinforce vertices that are likely to appear in large solutions [8]. The k-MDB problem structure lends itself well to this algorithm: an ant can grow a near-biclique one vertex at a time, checking at each step whether the defect budget permits another addition. These considerations motivate our choice of ACO over alternative metaheuristics for identifying large k-DB.

To our knowledge, the θ-heuristic of Cui et al. [1] is the only published heuristic algorithm for the k-MDB problem. ACO is therefore compared against the only existing heuristic baseline in the literature.

III. METHODS

A. ACO Algorithm

1) Overview of Algorithm: ACO traditionally uses a set of agents, called "ants", that traverse a graph and build a solution that must meet certain constraints. The ants leave pheromone on the vertices they visit, attracting other ants to follow their path. As the ants explore the solution space, they all converge on the same vertices as the quantity of pheromones on that path becomes increasingly larger.

We adapt the ACO algorithm to the k-MDB problem. A population of ants iteratively adds vertices from G to a subgraph S, preferring vertices with higher subgraph degree d_S, higher global degree d_G, and higher pheromone τ. Across ants and epochs, vertices repeatedly selected in high-quality solutions accumulate pheromone, increasing their likelihood of future selection.

We also add an optional control that improves practical runtime and performance. Flag F_N (neighbor-scope restriction) restricts candidates to the intersection of the k-feasible candidate set and the last-added vertex's neighbors when that intersection is nonempty, substantially reducing the set of vertices an ant must score at each step.

As an example, consider an ant has constructed S and its last added vertex is v_{last}. If candidates u_1, u_2, u_3 preserve the k-defect bound but only u_1, u_2 are adjacent to v_{last}, the neighbor-scope rule restricts the ant to $\{u_1, u_2\}$. It then samples between them according to their pheromone and desirability scores, adds the selected vertex, and repeats until no feasible candidate remains. F_N enables this flag and is our default for all experiments unless noted otherwise.

2) Algorithm Pseudocode: Algorithm 1 implements the ant colony loop with optional neighbor-scope restriction F_N. Here, $\sigma(x) = 1/(1+e^{-x})$ denotes the standard logistic sigmoid, used to dampen the impact of a vertex's degree with the entire graph as the size of the subgraph grows. Algorithm 2 implements desirability.

Algorithm 1 ACO

Require: A bipartite graph G; hyperparameters T, A, F_N, ρ, and $\Delta\tau$; parameters k and θ
Ensure: a k-MDB of G, if one could be found

```
function ACO(G, T, A, F_N, ρ, Δτ) ▷ F_N: neighbor-scope flag
    for all u ∈ G do
        τ(u) ← 1
    B ← {}
    for e ← 1, ..., T do
        S ← A empty subgraphs
        τ_C ← τ
        parallel for each S_i ∈ S do
            C ← {u ∈ G s.t. |Ē(S_i ∪ {u})| ≤ k}
            while |C| > 0 do
                if S_i = ∅ then
                    search ← C
                else
                    v_last ← the last vertex that was appended to S_i
                    C_{v_last} ← N(v_last)
                    if |C ∩ C_{v_last}| > 0 and F_N then
                        search ← C ∩ C_{v_last}
                    else
                        search ← C
                next ← linear sample of u ∈ search based on DESIRABILITY(u, S_i, τ_C)
                add next to S_i
                τ_C(next) ← τ_C(next) + Δτ ▷ updates to τ_C are atomic
                C ← {u ∈ G | k ≥ |Ē(S_i ∪ {u})|}
        S_B ← argmax_{S_i ∈ S} |E(S_i)| ▷ S_B is guaranteed to have at least one vertex
        if E(S_B) > E(B) then
            B ← S_B
        τ ← τ_C
        for all u ∈ G do
            τ(u) ← τ(u) × (1 − ρ)
    return B
```

We introduce a desirability function that prefers vertices with higher d_S, higher d_G, and higher pheromone τ.

Algorithm 2 Desirability

```
function DESIRABILITY(next, S, τ)
    η ← d_S(next) + d_G(next) × (1 − σ(|U_S| + |V_S|))
    return τ(next) × η^3
```

The η exponent was ablated over $\{1, 2, 3\}$, and 3 demonstrated the best quality.

We also tried elitist pheromone deposition, local tabu search, the MAX-MIN Ant System [4], a genetic algorithm adapted from GCLIQUE [3], and a prefer-smaller-

side desirability bias (flag F_P) that multiplies η by a constant when exactly one side of the ant's subgraph has reached θ and the candidate lies on the deficient side. None of the first four outperformed base ACO; the genetic algorithm failed on large graphs, indicating that it struggles with the dimensionality of large bicliques. Alone, F_P improves solution quality over plain ACO, but once neighbor-scope restriction F_N is enabled it contributes nothing further (Section 4.3).

3) Validity Guarantee: An ant iteratively searches over a set of vertices C such that any vertex $v \in C$ results in $\leq k$ missing edges if added to S. Flag F_N considers the intersection of this set with another. In either case, an ant is guaranteed to only add vertices that adhere to the k constraint. An ant is not guaranteed to return a k-DB with $\geq \theta$ vertices on both sides, even on graphs where such a k-DB exists, which is a limitation of the θ-heuristic as well. However, we observe that ACO is more likely to return θ-feasible solutions than the θ-heuristic (Section 4.3).

4) Theoretical Time Complexity: Here we analyze the upper complexity bound of our ACO implementation.

Subgraph Exploration Throughout this analysis, $n = |U| + |V|$ and $m = |E|$ refer to the graph on which ACO runs. In our experiments, ACO runs on graphs that have undergone common-neighbor reduction (Section 3.2), rather than the original graph. For each ant, it must first examine n vertices. At every subsequent step, it will need to consider all vertices it can add to its subgraph without violating the k constraint. A naive approach would examine $n-1$ vertices, and then $n-2$ vertices, etc. Thus, an ant will examine at most $n + (n-1) + (n-2) \ldots = n(n+1)/2$.

Upper Bound The upper time complexity would therefore be $\mathcal{O}(A \times T \times n^2)$ for A ants advancing over T epochs. In a parallelized implementation with p processors, the true upper complexity bound is $\mathcal{O}(\lceil A/p \rceil \times T \times n^2)$.

5) Practical Complexity: In practice, however, there are two factors limiting this complexity.

Neighbor-Scope Restriction First, let v_{last} be the last vertex an ant visited. F_N often only visits $d_{v_{last}}$ vertices each step (Section 3.1.1). Empirically, this means the ant will visit a number of vertices equal to the graph's average degree, $d^{\text{avg}} = 2m/n$, rather than the worst-case maximum vertex degree. This motivates a practical scaling model of $\mathcal{O}(\lceil A/p \rceil \times (T \times n \times d^{\text{avg}} + n))$. Note that $n \times d^{\text{avg}} = n \times \frac{2m}{n} = 2m$. The practical complexity therefore reduces to $\mathcal{O}(\lceil A/p \rceil \times (T \times m + n))$. On sparse graphs where $m \approx \mathcal{O}(n)$, this empirical model scales linearly with n, though dense graphs may still trigger the worst-case scenario of n^2.

Practical Candidate Set Size Additionally, while the upper bound of the candidate set pass is $\mathcal{O}(n)$, we optimize this pass to remove vertices from a memoized candidate set at each step. This results in an optimized complexity of $\mathcal{O}(|U_C| + |V_C|)$, where C is the set of candidate vertices before it advanced. If C remains very large, this could still be $\mathcal{O}(n)$ but practically runs much faster.

To check that observed runtimes follow these bounds, Figure 3 plots F_N discovery time normalized by ant count and epoch budget, $t/(A \cdot T)$, on log–log axes against each candidate complexity scale. For both scales, we fit the line $\log\big(t/(A \cdot T)\big) = \alpha \log(\text{bound}) + c$. A complexity scale proportional to runtime should give slope $\alpha \approx 1$. The naive n^2 slope falls well below 1 at 0.43, confirming that the quadratic worst case overestimates growth, while the practical bound $T \cdot m + n$ with $T = 5$ lands closer to 1 at 0.76, supporting it as the better complexity scale.

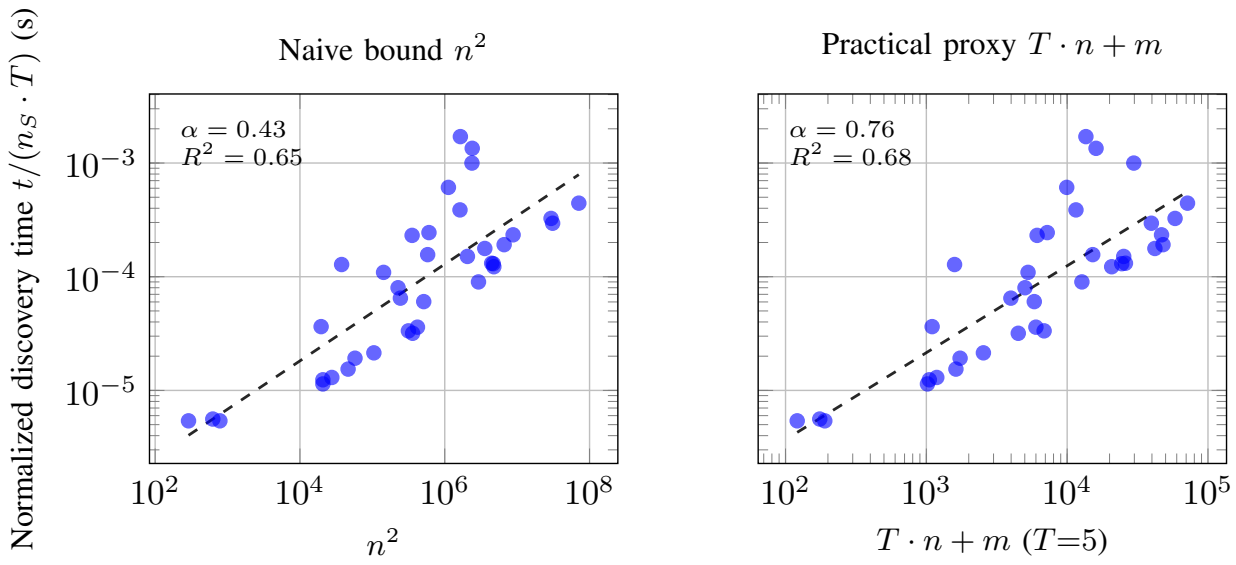


Fig. 3. F_N discovery time normalized by ant count and epoch budget $t/(n_S \cdot T)$ on the 37 $(k, \theta) = (2, 5)$ graphs (log–log; $n_S = 100$, $T = 5$ from recorded trials). Left: vs. the naive theoretical bound n^2 (slope $= 0.43$, $R^2 = 0.65$). Right: vs. the practical proxy $T \cdot n + m$ (slope $= 0.76$, $R^2 = 0.68$). Fitting $\log\big(t/(n_S \cdot T)\big) = \alpha \log(\text{bound}) + c$, slope ≈ 1 would indicate proportional growth. The practical slope is closer to 1 than the naive n^2 fit, so while still loose the practical bound is a significantly better complexity scale.

B. Branch-and-Pivot Approach

Finally, we implement the approach proposed by Cui et al. These authors observe that many θ-feasible k-DB have one side with exactly θ vertices. They therefore propose a heuristic that iteratively expands one side of the graph to θ vertices while removing vertices on the other side to respect the k constraint. We summarize their algorithm below.

Algorithm 3 θ-Heuristic

Ensure: a k-defective biclique of G

```
function ThetaHeuristic(G)
    U ← {}
    V ← V_G
    while |U| < θ do
        append u ∈ U_G \ U with largest d_V to U
        while |Ē(U ∪ V)| > k do
            remove v ∈ V with largest d̄_U from V
    return U ∪ V
```

The θ-heuristic has a time complexity of $\mathcal{O}(\theta n + m)$ [1].

They then propose a branch-and-pivot algorithm seeded with this initial heuristic lower bound. The algorithm checks combinations of $S = (U_S, V_S)$ by maintaining a candidate set $C = \{u \in G \text{ s.t. } |\bar{E}(S \cup \{u\})| \leq k\}$ of all remaining vertices that could still be added to S. It recursively chooses a vertex $u \in C$ and branches on $(S' = S \cup \{u\}, C' = \text{candidate set of } S')$ and $(S' = S, C' = C \setminus \{u\})$. See Appendix C for full pseudocode.

The paper also proposes a graph reduction technique called common-neighbor reduction. Common-neighbor reduction prunes edges between vertices v_1 and v_2 if v_1 has fewer than $\theta - k$ common neighbors with v_2, and prunes vertices with fewer than $\theta - k$ edges. This reduction technique can significantly reduce the number of vertices that branch-and-pivot, or other approaches such as the θ-heuristic and ACO, must search through.

C. Experimental Setup

Unless otherwise noted, all ACO experiments use the following hyperparameter values:

TABLE I
ACO HYPERPARAMETERS USED IN ALL EXPERIMENTS

Parameter	Value
Evaporation rate (ρ)	0.05
Pheromone deposit ($\Delta\tau$)	1
Number of epochs (T)	5
Number of replicates (R)	5
k	2
θ	5

The effect of T and R on performance is studied in Section 4.6. ρ was tuned manually by observing convergence behavior on a small subset of graphs. $\Delta\tau$ was set to a reasonable fixed value without a formal search. A more systematic hyperparameter study is left to future work.

An ACO *replicate* is an independent execution of the full ACO algorithm, including its own ant population and pheromone state. Unless otherwise noted, each replicate used $A = 100$ ants and $T = 5$ epochs. Because the implementation is stochastic, independent replicates can explore different solutions. Because $R = 5$, ACO was run six times; the first trial warmed up the Julia JIT compiler and took far longer than the others. It was discarded on all runs, and the last 5 trials were used.

For comparisons against the deterministic θ-heuristic, the reported ACO solution is the best solution obtained across these five independent replicates. Thus, comparisons measure the quality obtainable from a five-replicate ACO budget rather than a single ACO execution. Runtime comparisons include the cost of all five replicates. Section 4.6 examines the impact of replicate count on performance in more detail.

When testing ACO and the θ-heuristic, a biclique with k missing edges and θ vertices on each side was injected into each graph to make graphs without a naturally occurring k-DB usable for tests. Because the θ-heuristic was designed for $\theta \times q$ bicliques where $q \gg \theta$, this might unintentionally favor ACO (Section 5.1). Sometimes, graphs may have bicliques larger than the injection.

We test on 37 graphs from two public collections: 28 from KONECT (http://konect.cc), a standard repository of real-world bipartite networks, and 9 from Amazon Reviews 2023 (http://amazon-reviews-2023.github.io), user-product review graphs in selected product categories. All tests were run with 8 threads on a computer with Ubuntu, an Intel(R) Core(TM) Ultra 7 258V CPU, and 32 gigabytes of RAM.

First, we examine the effect of ant count on solution quality and runtime. We analyze ACO's performance compared to the θ-heuristic, demonstrating it can often find substantially larger solutions. We ablate neighborscope restriction F_N and prefer-smaller-side F_P, demonstrating that F_N makes ACO faster and improves quality. Additionally, ACO's improvements over the θ-heuristic can meaningfully reduce branch-and-pivot runtime. In a hyperparameter study, we examine how k and θ affect ACO performance compared to the θ-heuristic. We then analyze how ACO's wall-clock cost scales with these parameters. Finally, we examine how the number of epochs and independent replicates affect ACO solution quality.

IV. Results

We define an ACO win as an instance for which ACO produces a θ-feasible solution with more edges than the θ-heuristic. If the θ-heuristic is infeasible but ACO is feasible, ACO is also considered the winner. If both the ACO and θ-heuristic solution are θ-infeasible, the instance is counted as a tie. If the θ-heuristic is θ-feasible but ACO is infeasible, or the θ-heuristic solution has more edges than the ACO solution, the instance is counted as a loss.

A. Effect of ant count

We varied ant count A across $\{2, 5, 10, 20, 50, 100, 200\}$ using F_N on the full benchmark suite. At each ant count, T was set to 5 epochs.

Figure 4 summarizes the first quartile, median, and third quartile of percent deviation from the θ-heuristic, θ-feasibility rate, and wall-clock time. θ-infeasible outcomes are included in the deviation panel and scored as a -100% deviation from the θ-heuristic. Solution quality generally improves with more ants but often plateaus before $A = 200$. θ-feasibility generally rises but can also fluctuate. Finally, runtime scales roughly linearly with A on the log-scaled time panel, as expected by Section 3.1.4.

B. ACO vs. θ-heuristic

We compared F_N against the θ-heuristic on the benchmark suite, recording time to completion, epochs to the best solution (ETB), and time to the best solution (TimTB). A complete definition of metrics is described in Appendix A. On 23 of 37 graphs, ACO produced a larger k-DB than the θ-heuristic, underperforming on 5 and tying on 9. In these trials, ACO's median performance was a 76.9% edge increase over the θ-heuristic among θ-feasible outcomes, although it sometimes was several orders of magnitude slower than the θ-heuristic (Section 4.5.1). ACO had a θ-feasibility rate of 89.2% versus the θ-heuristic's feasibility rate of 67.6%.

Table II shows the largest relative F_N wins and the closest θ-heuristic wins. Full per-graph results appear in Appendix A.

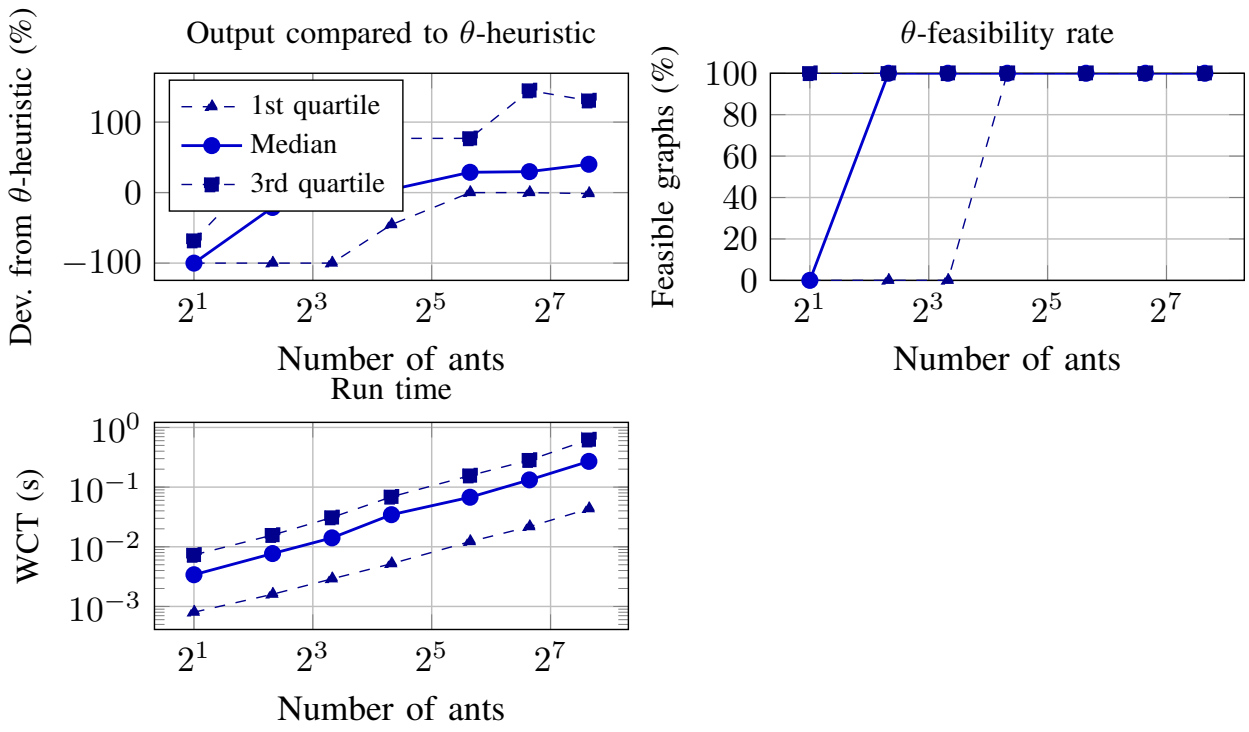


Fig. 4. F_N solution quality vs. the θ-heuristic, θ-feasibility rate, and wall-clock time vs. ant count. Each panel shows the cross-graph first quartile, median, and third quartile of per-graph values (not pooled replicate trials; JIT warmup omitted). θ-infeasible outcomes count as -100% deviation from the θ-heuristic. Quality and feasibility generally rise with colony size but often plateau before the largest n_S; runtime scales roughly linearly.

TABLE II
REPRESENTATIVE F_N VS. θ-HEURISTIC COMPARISONS ($k = 2, \theta = 5$). TOP: LARGEST RELATIVE $|E(D_{best})|$ GAINS FOR F_N. BOTTOM: CLOSEST θ-HEURISTIC WINS (SMALLEST EDGE DEFICITS). FULL RESULTS ARE IN TABLE V.

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	$\lvert E_R\rvert$	F_N						θ-Heuristic			
							$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Discovery	ETB	TimTB	$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Time
Fashion	2035490	874297	2500962	88	78	356	5	5	23	6.5	1	1.1	5	1	3	0.1
Wiki Talk Nds	605	23094	27455	45	149	616	7	5	33	64.0	1	6.2	5	1	3	0.1
Matter	16726	22015	58618	2283	3276	11825	8	11	87	147.7	4	108.7	5	4	18	1.3
Edit Nowikinews	430	4060	16923	82	1192	5177	5	33	163	193.5	2	76.7	5	9	43	0.8
Moreno Blogs	1065	990	19048	809	625	18044	74	5	368	75.4	5	70.7	5	3	13	1.0
P2P Gnutella09	3055	8038	26036	455	106	4056	4	4	14	16.7	1	3.1	5	5	23	0.3
Instruments	1762679	259791	3017462	6279	2153	29397	4	4	14	221.8	1	43.0	5	7	33	3.4
Dblp Cite	3158	11566	49782	2466	2955	31279	5	13	63	162.8	3	95.2	5	18	88	2.8

C. Flag ablation (F_P and F_N)

We evaluated ACO, F_P, F_N, and F_{PN} on all benchmark graphs with 100 ants. Figure 5 summarizes the solution quality and time for all four algorithms. Quality is the per-graph mean of ACO edge counts, taking the best solution across all 5 ACO replicates. In cases where the best solution was θ-infeasible, the graph counts as 0 edges but remains in the mean. Figure 6 reports the corresponding graph-level θ-feasibility rates, the percentage of graphs where ACO with the included flags identified a θ-feasible solution.

Neighbor-scope restriction F_N cuts discovery time versus plain ACO. Adding prefer-smaller-side F_P on top of F_N (F_{PN} vs. F_N) yields no further gain, and F_P alone remains slower and weaker than F_N. Neighbor-scope restriction also changes how ants spend the defect budget early in construction. Indeed, we find that with F_N, the average number of missing edges after a subgraph has reached 5 vertices is 0.60, much smaller than the average of 1.11 edges using plain ACO. Prefer-smaller-side F_P alone does not produce the same effect; once F_N is enabled, F_P adds no further quality or runtime benefit over F_N (Figure 5).

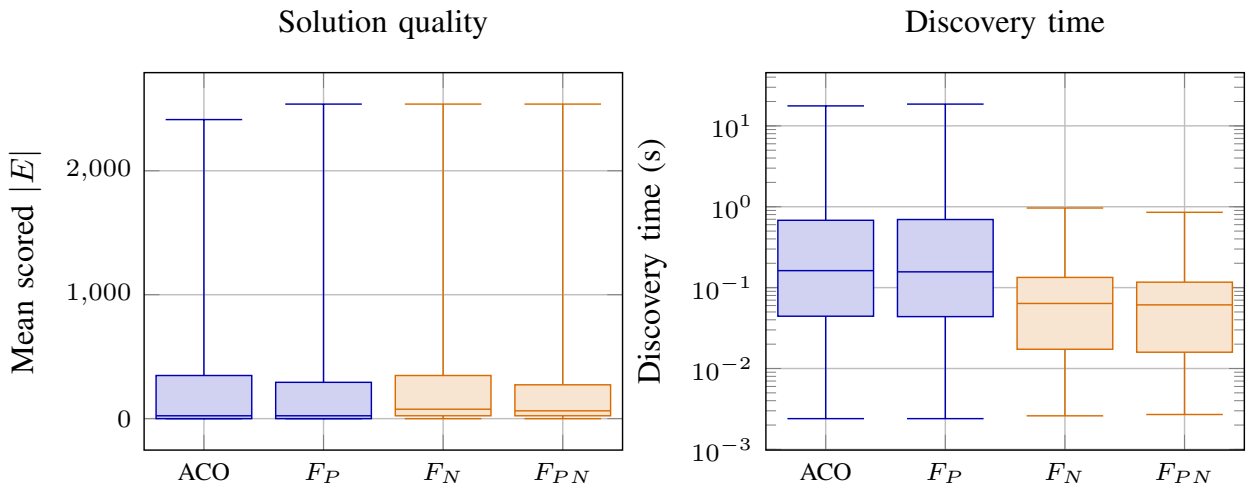


Fig. 5. Flag ablation at 100 ants on 37 matched graphs. Left: per-graph mean scored $|E|$ over counted replicates (θ-infeasible trials count as 0 edges but stay in the mean). Right: per-graph discovery time (sum of counted replicate wall times). Both panels are ordered and colored by neighbor-scope F_N.

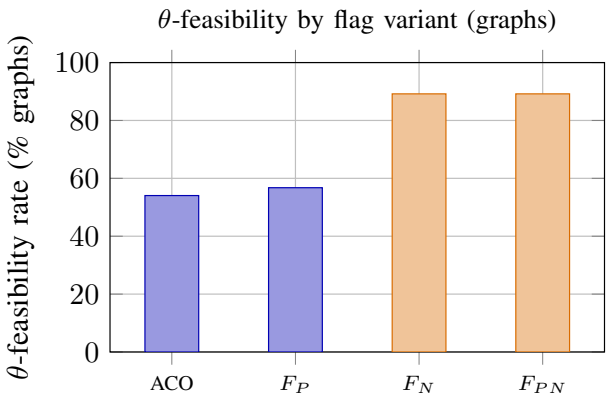


Fig. 6. Percentage of 37 matched graphs with at least one θ-feasible counted replicate at 100 ants. Variants are ordered ACO, F_P, F_N, F_{PN} and colored by neighbor-scope F_N.

D. Branch-and-pivot seeding

A better initial solution D, and therefore a larger lower bound $|E(D)|$, can allow branch-and-pivot to prune more branches and sometimes reduce runtime substantially. In graphs where ACO produced a θ-feasible solution with more edges than the θ-heuristic, its returned k-DB was used as a seed. Branch-and-pivot was then run twice on each eligible graph: once seeded with the θ-heuristic solution and once seeded with the ACO solution, omitting graphs where both runs timed out as described in Section 3.3. In cases where the θ-heuristic produced a θ-infeasible solution, the lower bound of edges was simply the smallest possible number of edges in a k-MDB, $\theta^2 - k$.

Table III shows the graphs where ACO seeding reduced total wall-clock time, together with the smallest and largest overhead cases. Net *time saved* is defined in Appendix B, where positive values mean the ACO-seeded pipeline is faster overall. The largest absolute savings occur on the largest graphs where a pure θ-heuristic-seeded pivot is expensive. When ACO does not improve D_{best}, the extra search cost remains small relative to the pivot wall-clock time. Full per-graph results are shown in Table XI (Appendix B).

TABLE III
REPRESENTATIVE PIVOT SEED COMPARISONS. TOP: GRAPHS WHERE ACO SEEDING REDUCED WALL-CLOCK TIME. BOTTOM: SMALLEST AND LARGEST ACO-SEARCH OVERHEAD AMONG GRAPHS WHERE ACO DID NOT BEAT THE θ-HEURISTIC. FULL RESULTS IN TABLE XI.

Dataset	$\lvert U_G\rvert + \lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert + \lvert V_R\rvert$	$\lvert E_R\rvert$	Time saved (s)	Reduction (%)
Edit Jamwiki	2918	17669	1548	8298	6.26	6.52
Edit Zuwiktionary	1618	7555	595	3142	51.61	49.45
Edit Guwikiquote	2641	11387	778	3328	152.02	54.42
Edit Nowikinews	4490	16923	1274	5177	291.36	30.37
Edit Vewiki	2075	11069	378	3385	714.47	56.15
Music	171471	130457	144	298	-0.01	-0.00
Edit Wowiktionary	3839	17800	1282	7175	-8.45	-0.66

E. Variations in k and θ

The main experiments fix $k = 2$ and $\theta = 5$. To demonstrate that F_N's advantage over the θ-heuristic holds for varied k and θ, we reran the 100-ant comparison for $k \in \{1, 2, 3, 4\}$ at $\theta = 5$, and for $\theta \in \{5, 6, 7\}$ at $k = 3$.

1) Solution Quality across k and θ: Figure 7 holds $\theta = 5$ and varies k. Relative ACO edge counts and ACO win rate compared to the θ-heuristic do not collapse as the defect budget grows. Figure 8 holds $k = 3$ and varies θ.

We observe that ACO performs better with larger k and smaller θ. This may be explained by differences in density among the graphs. Figure 9 plots each ACO win/loss for each graph on each (k, θ) setting against the reduced graph's density $\text{den}(G_R) = |E_R|/(|U_R|\,|V_R|)$, overlaid with a sliding-window win-rate curve (W = 21). We observe that ACO wins more often on sparser graphs. Figure 10 shows that common-neighbor reduction leaves graphs denser with larger k and sparser with larger θ, so density is a plausible correlate with ACO success. Together, these sweeps suggest that ACO's advantage over the θ-heuristic is not confined to a single (k, θ) setting. Future work might explore more extensive sweeps to further confirm this hypothesis.

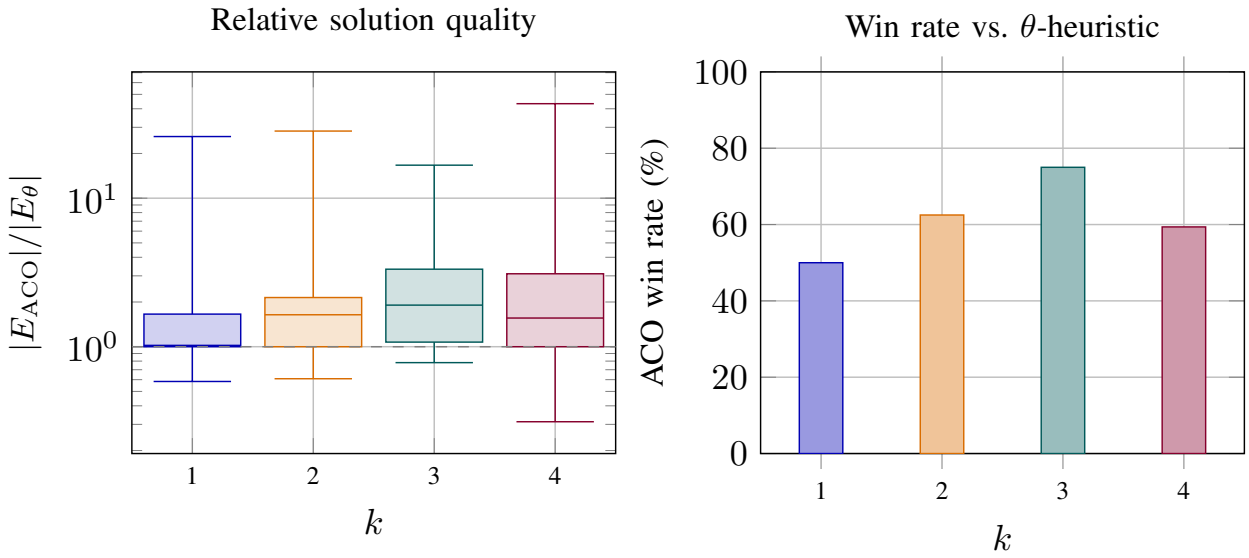


Fig. 7. Effect of defect budget k at fixed $\theta = 5$ on 32 graphs matched across all shown settings (100 ants, F_N). Left: edge ratio $|E_{\mathrm{ACO}}|/|E_\theta|$ (log scale; dashed parity). Right: ACO win rate vs. the θ-heuristic. Relative edge counts hold as k grows, and ACO wins more often at larger k.

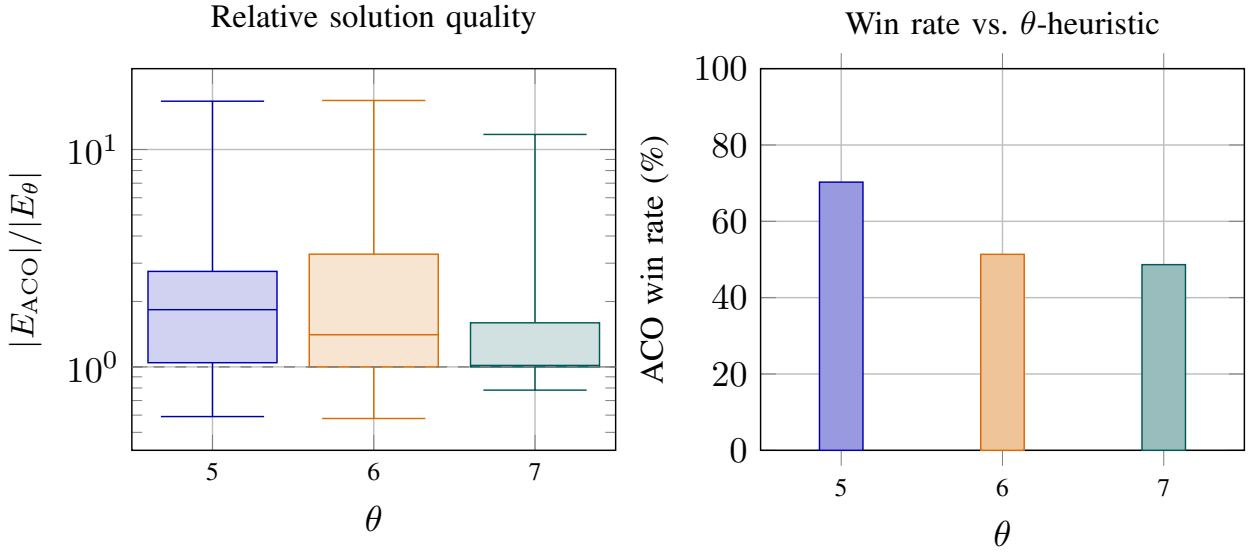


Fig. 8. Effect of minimum side size θ at fixed $k = 3$ on 37 graphs matched across all shown settings (100 ants, F_N). Left: edge ratio $|E_{\mathrm{ACO}}|/|E_\theta|$ (log scale; dashed parity). Right: ACO win rate vs. the θ-heuristic. Raising θ modestly lowers relative edge count and win rate, but ACO still beats the heuristic on a clear majority.

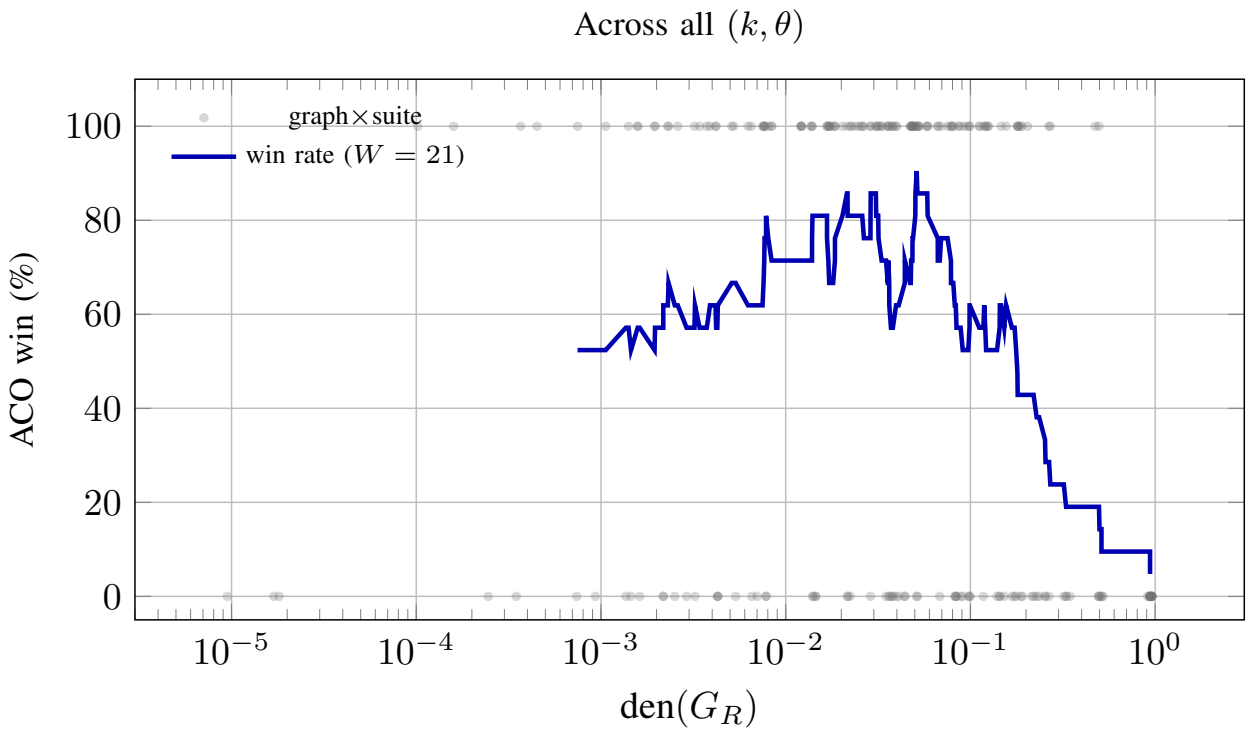


Fig. 9. F_N vs. the θ-heuristic (100 ants) on every graph×suite observation ($n = 217$). Marks are binary outcomes (win = 100%, else 0%). The curve is a sliding-window win rate over $W = 21$ consecutive observations sorted by reduced edge density den$(G_R) = |E_R|/(|U_R|\,|V_R|)$ (x = window-median density). Win rate falls as reduced density rises.

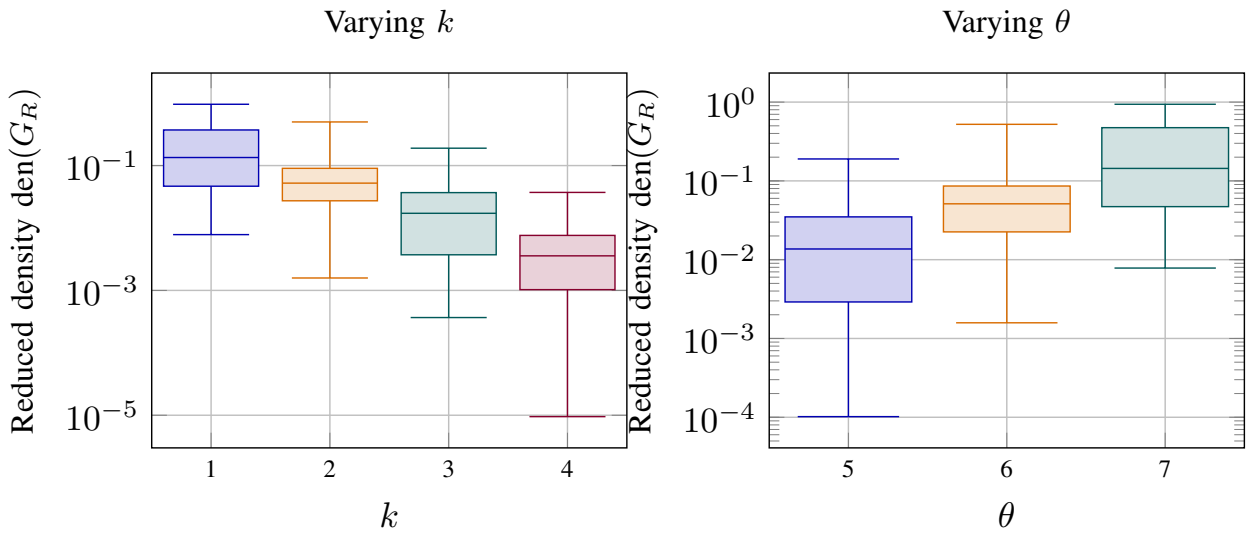


Fig. 10. Reduced edge density den$(G_R) = |E_R|/(|U_R|\,|V_R|)$ (log scale) after common-neighbor reduction (threshold $\theta - k$). Left: vary k at fixed $\theta = 5$. Right: vary θ at fixed $k = 3$. Raising k tends to leave sparser reduced graphs, while raising θ tends to leave denser ones, so density—not θ alone—tracks the win-rate shift in Figure 9.

2) Runtime across k and θ: Figure 11 compares F_N discovery time to the θ-heuristic on the same (k, θ) sweeps. On graphs that ACO completed under the ACO wall-clock budget, ACO takes a median of $400\times$–$700\times$ times longer, depending on k and θ. Median absolute ACO discovery time, however, generally remains under two seconds. Raising k at fixed $\theta = 5$ tends to increase ACO search time; raising θ at fixed $k = 3$ tends to decrease it, likely because smaller k and larger θ allow common-neighbor reduction to prune more vertices and edges.

At $k = 4$, 10 of 37 graphs timed out under a $600\,\mathrm{s}$ budget, and these graphs were excluded from the solution quality and discovery time graphs. Therefore, ACO might have a higher win rate than was recorded, although at the expense of significant runtime.

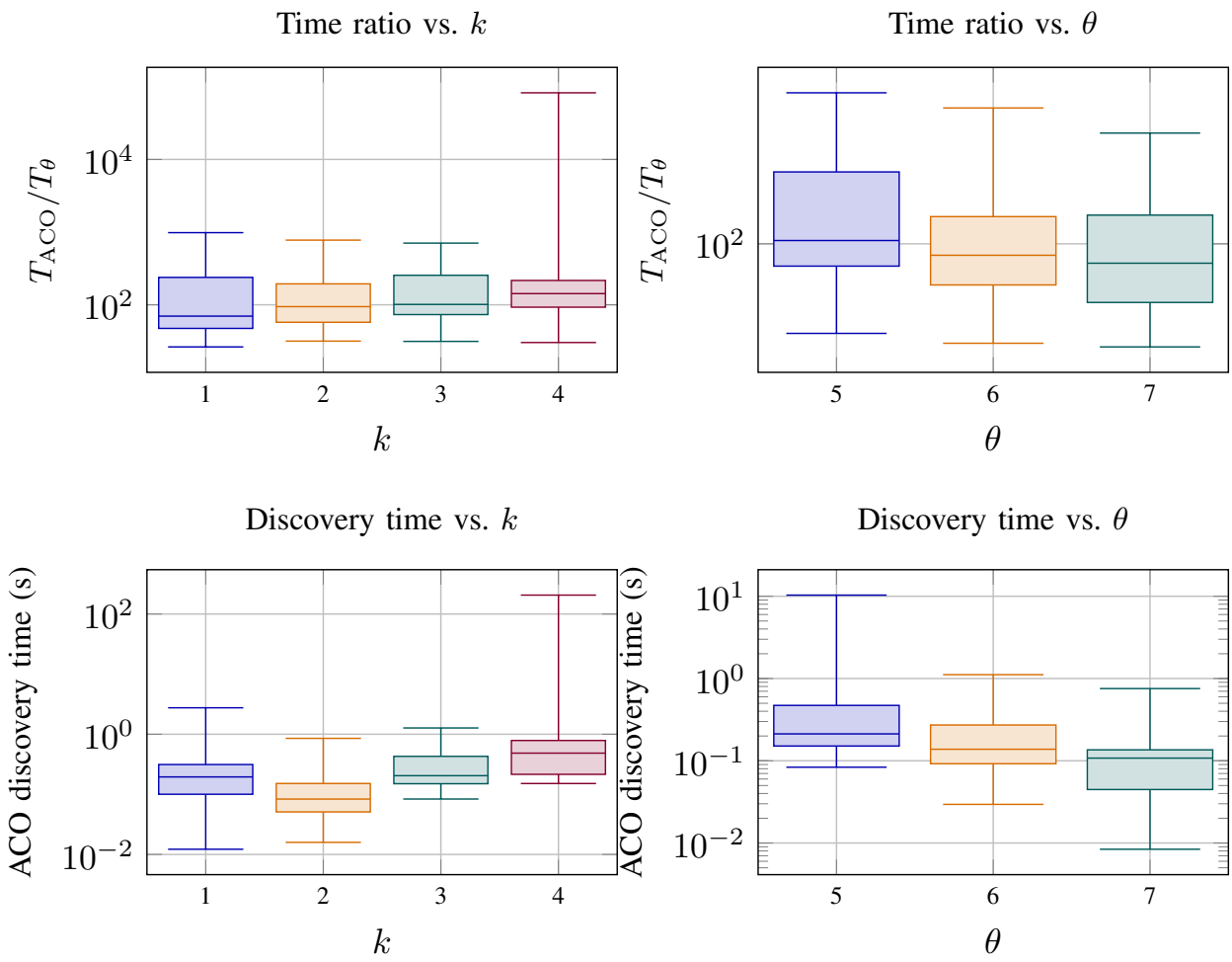


Fig. 11. F_N discovery cost versus the θ-heuristic (100 ants) across (k, θ) suites. Top: ratio $T_{\mathrm{ACO}}/T_\theta$ (log scale). Bottom: absolute ACO discovery time (log scale). Left: vary k at fixed $\theta = 5$. Right: vary θ at fixed $k = 3$. ACO is typically hundreds of times slower than the heuristic but still generally has a runtime of a few seconds. Increasing k lengthens search while increasing θ shortens it, likely because a low k and large θ enable common-neighbor reduction to prune more nodes and reduce the search space. Timed-out graphs are excluded.

F. Effect of epoch and replicate budget

Holding ants fixed at $A = 100$, we vary the number of epochs per ant and the replicate count. For each epoch budget T, ACO is run for exactly T epochs; for each replicate budget R, we use the first R independent replicates run on the graphs from Section 4.2 and take the best among them. Figures 12 and 13 each show the percentage of graphs where ACO beat the θ-heuristic found a θ-feasible solution; mean and median ACO percent edge increase compared to the θ-heuristic among graphs where it found a θ-feasible solution; and the cumulative distribution function (CDF) of when the eventual best was reached. The CDF measures the probability that the best solution on a graph would have been found within T epochs (Figure 12); and the probability that the best solution on a graph would have been found within R replicates (Figure 13).

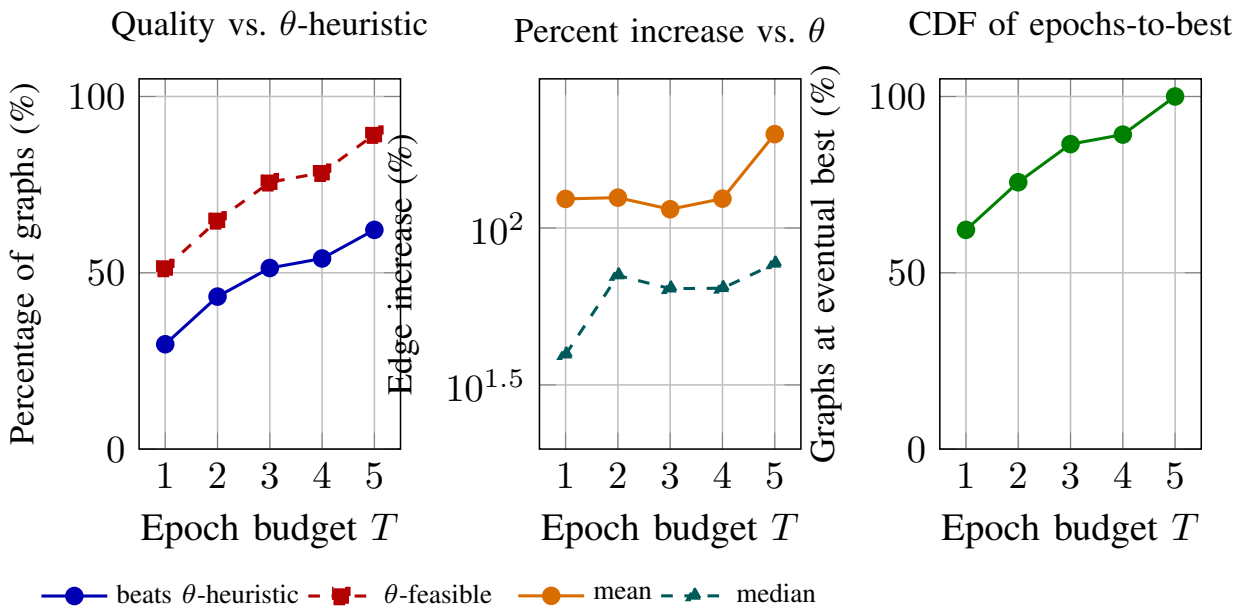


Fig. 12. Retrospective epoch-budget analysis at 100 ants (k=2, θ=5, $T \in \{1, \ldots, 5\}$). Most of F_N's advantage over the θ-heuristic requires few epochs. Win rate and θ-feasibility largely plateau by 3 epochs. Median and mean edge increase slightly after 3 epochs, but the majority of increase happens in the first 3 epochs. Indeed, by epoch 3, 86.5% of graphs have already reached their best edge count over all counted replicates.

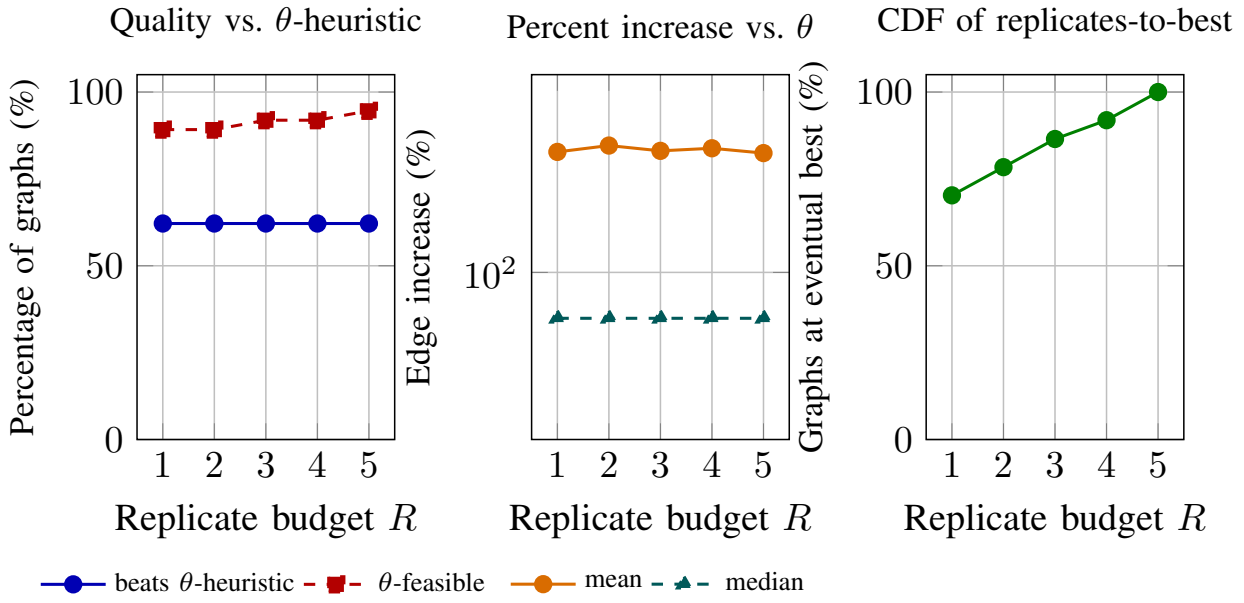


Fig. 13. Replicate budget at 100 ants (k=2, θ=5; best among the first R counted replicates, up to $R_{\max}$=5). A single counted replicate already recovers essentially all head-to-head wins; additional seeds mainly improve late replicates-to-best coverage and slight θ-feasibility gains rather than new wins against the θ-heuristic.

V. Discussion

A. Interpretation of Results

While ACO is substantially slower than the θ-heuristic, it often finds solutions several times larger in edge count. ACO's relative cost is not fixed. Ant count A, epoch budget T, and replicate count R can be adjusted for a linear effect on wall-clock time (Sections 4.1 and 4.6). Reducing either A, R or T shrinks discovery cost roughly proportionally, at the expense of

some solution quality and θ-feasibility. Increasing them improves quality. Our reported ACO runtime is therefore a consequence of our hyperparameters ($A = 100$, $T = 5$, $R = 5$), rather than a necessary cost of ACO.

In some cases, marginal quality gains can be worth it. Even small improvements ACO makes over the θ-heuristic can meaningfully reduce the time that branch-and-pivot takes. On the 5 graphs where ACO beat the θ-heuristic (Table III), ACO-seeded runtimes can significantly reduce runtimes relative to a θ-seeded pivot. Additionally, ACO had a higher θ-feasibility rate than the θ-heuristic, meaning it can be used as a valid lower bound more often. This suggests that ACO provides a substantially better lower bound to use for branch-and-pivot than the θ-heuristic in certain cases.

ACO improves quality in several types of solutions. On several graphs, the θ-heuristic returns a θ-feasible biclique with exactly θ vertices on one side, the structural family it was designed for. In many of these instances, ACO finds a substantially larger opposite side while still keeping one side at exactly θ (e.g., Edit Jamwiki, 5×377 vs. 5×184). In several cases, the ACO heuristic identified $\theta \times \theta$ k-DB on graphs that the θ-heuristic failed to make θ-feasible (e.g., Appliances). Other wins are entirely different. ACO may escape the θ-exact limitation (e.g., Dnc Corecipient, 62×10 vs. 5×89).

We hypothesize this is due to ACO's exploration. Because the θ-heuristic arbitrarily chooses U as its θ-side and greedily expands, early vertex choices can permanently shrink the retained neighborhood. Conversely, ACO's stochastic, massively parallel construction samples alternative $\theta \times \theta$ subgraphs and often recovers a better one. Even on the graphs it was designed for, the θ-heuristic's greedy construction can be dominated by ACO search, which iteratively refines the θ vertices it uses.

These promising results are likely driven by F_N. We hypothesize that F_N shrinks the candidate set to a smaller pool of locally promising vertices, making ACO's stochastic sampling likelier to exploit promising options. However, we do not test this hypothesis, and confirmation is left to future work.

On the k and θ sweeps (Figures 7, 8), F_N's advantage over the θ-heuristic is not confined to $(k, \theta) = (2, 5)$, but its win rate does change across k and θ. This apparent dependence may be a result of reduced graph density, however. ACO wins more often on sparser reduced graphs, and variations in k and θ that altered win rate also altered reduced density. Thus, ACO is not necessarily better on larger k or lower θ.

Figure 12 shows that ACO-F_N's win rate against the θ-heuristic rises steadily as the number of epochs increases. Additionally, Figure **??** shows that increasing the number of replicates, which reinitialize pheromone, does not have a significant impact on ACO win rate. This indicates that the pheromone system is conducive to ACO solution quality, providing an effective signal for what vertices are likely to appear in a large k-DB.

B. Limitations

As described in Section 3.3, our evaluation of ACO-seeded branch-and-pivot was limited to the graphs that were completed under the timeout. Branch-and-pivot's runtime grows exponentially with graph size even under a tight bound, and running it to completion on our larger test graphs (e.g., Matter, Dimacs Polblogs) was not feasible on the hardware available for this study. As a result, our seeding experiment cannot yet confirm whether the pruning benefit observed on small graphs extends to larger instances. However, ACO seeding may scale to larger graphs.

More broadly, because branch-and-pivot runtime scales exponentially with vertex count, we could not validate ACO's solution quality against the absolute optimum on large graphs. These large graphs are where a quick-running heuristic such as ACO is especially useful. Therefore, we do not claim that ACO produces near-optimal k-MDB solutions.

Finally, we injected bicliques with θ vertices on both sides into the graphs. Where a planted $\theta \times \theta$ biclique is present, the θ-heuristic may have performed better with a more realistic biclique of $\gg \theta$ vertices on one side. Therefore, the cases in which ACO beats the θ-heuristic on subgraphs with $\gg \theta$ vertices on only one side are better evidence that it is inherently a more explorative heuristic.

VI. Conclusion

Ant Colony Optimization is a promising heuristic approach to identifying large k-DB. As an emerging field, there do not exist many tested heuristics for this problem. The θ-heuristic, the only previously proposed heuristic in the literature, is significantly faster than our algorithm but often does not produce solutions as large. ACO offers an easily tunable quality–runtime tradeoff. One instance in which extra quality is worth extra time is in branch-and-pivot, where ACO can function as a much more effective lower bound, cutting total wall-clock-time by up to 56% on the tested instances.

A. Future Work

A natural next step would be to rerun the seeded branch-and-pivot comparison of Section 4.2 on higher-performance hardware, so that the larger graphs excluded under the timeout in Section 3.3 can be included and the scaling of the seeding benefit can be tested directly.

Additionally, ACO could be adapted to have N subspecies that find N solutions in parallel by using a repulsion mechanism similar to that proposed for diversity in

other ant colony variants [9], extending previous multi-colony parallelization strategies [10]. This could be an additional strength over the θ-heuristic, taking advantage of its stochasticity to identify multiple solutions.

1) Improvement of F_N: We identified several potential improvements to the F_N algorithm.

- It may be possible to use upper bounding techniques proposed by [1] to stop an ant if it provably cannot beat the current best solution.
1 proves that on a k-DB with at least k vertices on both sides, the maximum distance between two vertices is 3. Therefore, we conjecture that when the intersection of an ant's neighbors and the candidate set is null, an ant must only backtrack by a maximum of 3 vertices and consider those nodes' neighbors, rather than examining the entire candidate set. A complete proof or disproof is left to future work.
- Seeding ant exploration with heuristically determined vertices may further encourage exploitation of vertices likely to be present in the final solution.
- A hyperparameter ablation study (including $(\Delta\tau, \rho) = (0, 0)$ to ablate the pheromone system) could identify hyperparameter settings that further improve ACO, or identify which hyperparameter settings work best for which types of graphs
- The ACO algorithm could be adapted to run on a parallelized GPU. Without a pheromone system, thousands of ants could run in parallel without the need for a thread-blocking global update.

Code is available on https://github.com/pooch63/aco-kmdb.

REFERENCES

[1] D. Cui, R. Li, Q. Dai, H. Qin, and G. Wang, "On the Efficient Discovery of Maximum K-Defective Biclique," arXiv.org. Accessed: Aug. 11, 2026. [Online]. Available: https://arxiv.org/abs/2506.16121

[2] M. Dorigo, V. Maniezzo, and A. Colorni, "Ant System: Optimization by a Colony of Cooperating Agents," IEEE Transactions on Systems, Man and Cybernetics, Part B (Cybernetics), vol. 26, no. 1, pp. 29–41, 1996, doi: 10.1109/3477.484436.

[3] S. Shah, "GCLIQUE: An Open Source Genetic Algorithm for the Maximum Clique Problem," Aug. 2020, doi: 10.36227/techrxiv.12814217.v1.

[4] T. Stützle and H. H. Hoos, "– Ant System," Future Generation Computer Systems, vol. 16, no. 8, pp. 889–914, Jun. 2000, doi: 10.1016/s0167-739x(00)00043-1.

[5] A. E. Rizzoli, R. Montemanni, E. Lucibello, and L. M. Gambardella, "Ant Colony Optimization for Real-World Vehicle Routing Problems," Swarm Intelligence, vol. 1, no. 2, pp. 135–151, Sep. 2007, doi: 10.1007/s11721-007-0005-x.

[6] L. M. Gambardella, É. D Taillard, and M. Dorigo, "Ant Colonies for the Quadratic Assignment Problem," Journal of the Operational Research Society, vol. 50, no. 2, pp. 167–176, Feb. 1999, doi: 10.1057/palgrave.jors.2600676.

[7] K. A. Dowsland and J. M. Thompson, "An improved ant colony optimisation heuristic for graph colouring," Discrete Applied Mathematics, vol. 156, no. 3, pp. 313–324, Feb. 2008, doi: 10.1016/j.dam.2007.03.025.

[8] C. Solnon and S. Fenet, "A Study of ACO Capabilities for Solving the Maximum Clique Problem," Journal of Heuristics, vol. 12, no. 3, pp. 155–180, May 2006, doi: 10.1007/s10732-006-4295-8.

[9] W. Tfaili and P. Siarry, "A New Charged Ant Colony Algorithm for Continuous Dynamic Optimization," Applied Mathematics and Computation, vol. 197, no. 2, pp. 604–613, Apr. 2008, doi: 10.1016/j.amc.2007.08.087.

[10] M. Middendorf, F. Reischle, and H. Schmeck, "Multi Colony Ant Algorithms," Journal of Heuristics, vol. 8, no. 3, pp. 305–320, May 2002, doi: 10.1023/a:1015057701750.

[11] Z. Wang, L. Chang, and J. X. Yu, "Identifying Maximum Defective Bicliques in Large Bipartite Graphs," 2025 IEEE 41st International Conference on Data Engineering (ICDE), pp. 3710–3723, May 2025, doi: 10.1109/icde65448.2025.00277.

[12] H. Yu, A. Paccanaro, V. Trifonov, and M. Gerstein, "Predicting interactions in protein networks by completing defective cliques," Bioinformatics, vol. 22, no. 7, pp. 823–829, Apr. 2006, doi: 10.1093/bioinformatics/btl014.

[13] X. Chen, Y. Zhou, J.-K. Hao, and M. Xiao, "Computing maximum k-defective cliques in massive graphs," Computers & Operations Research, vol. 127, p. 105131, 2021, doi: 10.1016/j.cor.2020.105131.

[14] L. Chang, "Efficient Maximum k-Defective Clique Computation with Improved Time Complexity," Proc. ACM Manag. Data, vol. 1, no. 3, Art. 209, Sep. 2023, doi: 10.1145/3617313.

[15] K. Yu and C. Long, "Maximum k-Biplex Search on Bipartite Graphs: A Symmetric-BK Branching Approach," Proc. ACM Manag. Data, vol. 1, no. 1, Art. 49, Mar. 2023, doi: 10.1145/3588729.

APPENDIX

The tables below list per-graph results for F_N (100 ants, five epochs) against the θ-heuristic. Table V is the primary paper setting ($k = 2$, $\theta = 5$); subsequent tables cover the (k, θ) sweep from Section 4.5. $|U_G|$, $|V_G|$, and $|E_G|$ are the original graph sizes; $|U_R|$, $|V_R|$, and $|E_R|$

are the reduced graph sizes on which both methods run after common-neighbor reduction.

Rows are grouped into three blocks separated by horizontal rules—ACO wins, θ-heuristic wins, and ties or mutual θ-infeasibility—using the win rule in Section 4. Within each block, rows are sorted by ACO $|E(D_{best})|$.

For ACO, `Discovery` is the cumulative wall time through the winning replicate; `ETB` is the epoch at which that replicate first reached its best $|E(D_{best})|$; `TimTB` is the elapsed time within that replicate to reach ETB. $|U_{D_{best}}|$, $|V_{D_{best}}|$, and $|E(D_{best})|$ denote the returned biclique; all times are in milliseconds.

TABLE IV
$k = 1, \theta = 5$

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	$\lvert E_R\rvert$	F_N						θ-Heuristic			
							$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Discovery	ETB	TimTB	$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Time
Edit Mkwikisource	218	6635	13825	9	17	76	5	5	24	12.2	1	2.0	5	1	4	0.1
Edit Rowikibooks	443	4221	18550	25	66	308	5	5	24	22.2	3	12.8	5	4	19	0.3
Gift Cards	132732	1894	152434	28	21	120	5	5	24	10.5	1	2.0	5	4	19	0.1
P2P Gnutella09	3055	8038	26037	385	72	3461	5	5	24	42.5	1	8.3	5	4	19	1.0
Moreno Health	2313	2441	12993	39	28	199	6	5	29	14.3	1	2.7	5	5	24	0.1
Wiki Talk Nds	605	23094	27456	24	24	156	6	5	29	13.7	1	2.4	5	4	19	0.1
Instruments	1762679	259791	3017463	234	199	1481	8	5	39	41.5	2	16.7	5	7	34	0.8
Bitcoin	3286	3754	24210	583	622	11208	8	9	71	120.6	5	114.8	5	13	64	2.5
Gnutella	4935	10856	40018	254	139	1679	12	6	71	35.7	1	6.1	5	6	30	0.8
Matter	16726	22015	58619	389	488	2294	8	11	87	59.2	1	11.2	5	11	55	0.9
Edit Guwikiquote	963	1678	11388	86	401	2393	5	28	139	295.9	2	126.5	5	17	84	1.0
Ciaodvd	1438	4299	40157	842	1158	32262	35	5	174	194.9	5	181.4	5	20	99	7.4
Pgp Arenas	5596	9657	24340	574	743	7342	12	18	215	113.9	3	64.0	5	26	129	1.9
Petster Hamster	2244	1766	16655	967	628	11228	69	5	344	161.0	2	57.9	5	18	89	3.6
Moreno Blogs	1065	990	19049	704	500	17109	73	5	364	194.8	3	107.6	5	3	14	3.8
Edit Vewiki	543	1532	11070	95	231	3218	6	79	473	151.5	1	27.3	5	91	454	1.2
Dnc Corecipient	653	714	12109	342	326	11038	38	38	1443	212.8	3	120.0	5	88	439	3.4
Edit Jamwiki	177	2741	17670	50	1166	7275	5	376	1879	2136.7	1	619.5	5	183	914	3.3
Nawiki	844	4014	70719	169	1118	21246	10	253	2529	1686.2	5	1661.6	5	430	2149	7.7
Dblp Cite	3158	11566	49783	1630	1453	18547	5	10	49	277.1	5	269.8	5	17	84	5.0
Dimacs Polblogs	1224	1224	33454	847	848	31720	47	7	328	219.2	3	121.7	5	75	374	7.8
Openflights	3409	3418	67687	1177	1177	29982	7	63	440	259.5	4	192.0	5	91	454	9.6
Appliances	1755732	104237	2128629	5	5	24	5	5	24	7.5	1	1.1	5	5	24	0.0
Beauty	631986	115709	701552	9	9	40	5	5	24	8.0	1	1.2	5	5	24	0.0
Boxes	15237	1128	16240	5	5	24	5	5	24	7.6	1	1.2	5	5	24	0.0
Facebook	28	2885	3004	6	5	28	5	5	24	7.8	1	1.4	5	5	24	0.0
Fashion	2035490	874297	2500963	5	5	24	5	5	24	8.1	1	1.5	5	5	24	0.0
Health	461656	62597	494145	5	5	24	5	5	24	9.3	1	1.5	5	5	24	0.0
Maayan Vidal	1633	2929	6750	13	13	56	5	5	24	7.7	1	1.2	5	5	24	0.0
Magazine	60144	3391	71521	9	9	40	5	5	24	8.2	1	1.2	5	5	24	0.0
Moreno	888	1597	2301	5	5	24	5	5	24	7.6	1	1.1	5	5	24	0.0
Music	100952	70519	130458	5	5	24	5	5	24	7.7	1	1.3	5	5	24	0.0
Edit Srwikiquote	402	1823	8696	51	291	2142	5	24	119	125.9	2	51.8	5	24	119	0.9
Edit Nowikinews	430	4060	16924	50	708	3516	5	32	159	499.7	3	308.0	5	32	159	1.3
Edit Mrwiktionary	281	3995	14376	29	620	3378	5	93	464	1121.8	1	207.0	5	93	464	1.5
Edit Zuwiktionary	214	1404	7556	32	466	2848	5	117	584	364.6	1	82.2	5	117	584	1.1
Edit Wowiktionary	193	3646	17801	25	1018	6452	5	352	1759	2752.4	1	749.0	5	352	1759	2.8

TABLE V
$k = 2, \theta = 5$

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	$\lvert E_R\rvert$	F_N						θ-Heuristic			
							$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Discovery	ETB	TimTB	$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Time
Appliances	1755732	104237	2128628	1289	421	4175	5	5	23	45.0	1	8.3	5	3	13	0.6
Beauty	631986	115709	701551	103	137	536	5	5	23	9.6	1	1.7	5	3	13	0.1
Fashion	2035490	874297	2500962	88	78	356	5	5	23	6.5	1	1.1	5	1	3	0.1
Health	461656	62597	494144	57	87	330	5	5	23	6.2	1	1.0	5	2	8	0.1
Maayan Vidal	1633	2929	6749	81	134	547	5	5	23	7.7	1	1.4	5	3	13	0.1
Magazine	60144	3391	71520	248	76	922	5	5	23	10.7	1	1.9	5	3	14	0.2
Music	100952	70519	130457	63	81	298	5	5	23	5.7	1	1.0	5	3	13	0.1
Wiki Talk Nds	605	23094	27455	45	149	616	7	5	33	64.0	1	6.2	5	1	3	0.1
Moreno Health	2313	2441	12992	325	275	1496	6	6	34	15.9	2	5.8	5	6	29	0.2
Gnutella	4935	10856	40017	421	227	2758	9	7	61	18.0	2	6.4	5	6	30	0.3
Bitcoin	3286	3754	24209	1003	1105	15395	6	13	76	65.7	5	62.6	5	8	38	1.3
Matter	16726	22015	58618	2283	3276	11825	8	11	87	147.7	4	108.7	5	4	18	1.3
Edit Nowikinews	430	4060	16923	82	1192	5177	5	33	163	193.5	2	76.7	5	9	43	0.8
Ciaodvd	1438	4299	40156	978	1593	35082	35	5	173	95.7	3	49.5	5	13	63	2.4
Edit Guwikiquote	963	1678	11387	129	649	3328	5	35	173	122.2	1	21.1	5	18	88	0.5
Pgp Arenas	5596	9657	24339	936	1247	9791	10	22	218	61.2	3	33.7	5	27	133	1.0
Petster Hamster	2244	1766	16654	1310	852	13608	9	28	250	65.1	5	62.7	5	19	93	1.2
Moreno Blogs	1065	990	19048	809	625	18044	74	5	368	75.4	5	70.7	5	3	13	1.0
Edit Vewiki	543	1532	11069	104	274	3385	6	80	478	54.6	3	30.9	5	92	458	0.5
Edit Zuwiktionary	214	1404	7555	36	559	3142	5	118	588	115.5	2	51.6	5	49	243	0.5
Dnc Corecipient	653	714	12108	393	369	11379	62	10	618	78.2	1	8.9	5	89	443	1.0
Edit Jamwiki	177	2741	17669	64	1484	8298	5	377	1883	673.4	2	355.0	5	184	918	1.0
Nawiki	844	4014	70718	204	1337	22032	10	254	2538	499.5	1	93.9	5	431	2153	2.8
Instruments	1762679	259791	3017462	6279	2153	29397	4	4	14	221.8	1	43.0	5	7	33	3.4
P2P Gnutella09	3055	8038	26036	455	106	4056	4	4	14	16.7	1	3.1	5	5	23	0.3
Dblp Cite	3158	11566	49782	2466	2955	31279	5	13	63	162.8	3	95.2	5	18	88	2.8
Dimacs Polblogs	1224	1224	33453	945	946	32524	55	5	273	88.6	1	12.4	5	76	378	2.8
Openflights	3409	3418	67686	1480	1485	31970	6	59	352	116.8	5	107.8	5	92	458	2.9
Boxes	15237	1128	16239	14	14	50	5	5	23	2.7	1	0.4	5	5	23	0.0
Edit Rowikibooks	443	4221	18549	69	425	1515	5	5	23	32.4	1	7.4	5	5	23	0.2
Facebook	28	2885	3003	9	8	36	5	5	23	2.7	1	0.4	5	5	23	0.0
Moreno	888	1597	2300	13	12	50	5	5	23	2.8	1	0.5	5	5	23	0.0
Gift Cards	132732	1894	152433	620	96	2260	8	4	30	30.2	1	5.4	5	4	18	0.3
Edit Mkwikisource	218	6635	13824	19	121	399	4	9	34	18.2	1	3.0	5	2	8	0.1
Edit Srwikiquote	402	1823	8695	80	396	2637	5	25	123	40.0	1	7.2	5	25	123	0.3
Edit Mrwiktionary	281	3995	14375	46	1014	4630	5	94	468	305.3	1	59.0	5	94	468	0.7
Edit Wowiktionary	193	3646	17800	25	1257	7175	5	353	1763	853.7	1	245.5	5	353	1763	1.1

TABLE VI
$k = 3, \theta = 5$

Dataset	$\|U_G\|$	$\|V_G\|$	$\|E_G\|$	$\|U_R\|$	$\|V_R\|$	$\|E_R\|$	F_N						θ-Heuristic			
							$\|U_{D_{best}}\|$	$\|V_{D_{best}}\|$	$\|E(D_{best})\|$	Discovery	ETB	TimTB	$\|U_{D_{best}}\|$	$\|V_{D_{best}}\|$	$\|E(D_{best})\|$	Time
Boxes	15237	1128	16238	193	94	448	5	5	22	16.4	1	3.0	5	3	12	0.1
Health	461656	62597	494143	1904	1449	6405	5	5	22	143.4	1	28.3	5	3	13	1.5
Instruments	1762679	259791	3017461	147548	36728	551882	5	5	22	10327.4	5	9805.1	5	3	13	87.9
Maayan Vidal	1633	2929	6748	437	803	2823	5	5	22	52.0	4	40.3	5	3	13	0.6
Magazine	60144	3391	71519	3951	663	10198	5	5	22	229.7	2	84.9	5	2	8	2.2
Moreno	888	1597	2299	98	126	316	5	5	22	12.1	1	2.3	5	1	4	0.1
Facebook	28	2885	3002	12	79	180	5	5	23	31.2	1	4.5	5	1	3	0.1
Edit Mkwikisource	218	6635	13823	64	788	1812	5	5	24	188.1	5	186.7	5	3	12	0.5
Edit Rowikibooks	443	4221	18548	146	1666	4224	5	6	27	539.7	2	176.6	5	3	12	0.9
P2P Gnutella09	3055	8038	26035	1460	1174	7148	6	5	27	107.8	5	104.9	5	2	7	1.2
Wiki Talk Nds	605	23094	27454	126	757	1935	6	5	27	365.8	3	238.5	5	3	12	0.6
Moreno Health	2313	2441	12991	1665	1467	7832	7	6	39	121.5	3	70.9	5	3	12	1.3
Gnutella	4935	10856	40016	2208	2059	8866	11	5	52	171.5	3	100.2	5	1	4	1.8
Bitcoin	3286	3754	24208	1892	2083	20619	5	14	67	201.8	4	152.5	5	9	42	3.0
Matter	16726	22015	58617	7606	14142	39511	9	10	87	1150.8	4	876.1	5	3	12	7.9
Ciaodvd	1438	4299	40155	1171	2360	37784	33	5	162	212.3	4	152.7	5	14	67	4.3
Edit Nowikinews	430	4060	16922	161	1654	6312	5	34	167	304.3	2	120.9	5	10	47	1.5
Edit Guwikiquote	963	1678	11386	268	901	4109	5	37	182	206.8	3	118.7	5	19	92	0.8
Pgp Arenas	5596	9657	24338	1983	2815	14533	13	16	205	235.2	4	176.3	5	27	133	2.4
Petster Hamster	2244	1766	16653	1725	1203	15547	71	5	352	147.3	1	25.4	5	20	97	2.5
Moreno Blogs	1065	990	19047	927	759	18655	74	5	367	151.1	5	143.3	5	5	22	2.2
Edit Vewiki	543	1532	11068	167	486	3925	6	81	483	101.7	3	57.3	5	93	462	0.9
Edit Zuwiktionary	214	1404	7554	68	661	3380	5	119	592	177.5	1	38.6	5	50	247	0.7
Dnc Corecipient	653	714	12107	458	437	11647	57	16	909	134.9	1	17.2	5	90	447	1.8
Edit Jamwiki	177	2741	17668	93	2204	9795	5	328	1637	962.5	1	331.2	5	185	922	2.5
Nawiki	844	4014	70717	298	1988	23520	10	253	2527	818.4	2	293.7	5	432	2157	4.4
Gift Cards	132732	1894	152432	8743	597	22257	4	4	13	4774.0	3	1181.9	5	5	22	4.1
Dblp Cite	3158	11566	49781	2955	5734	42523	5	18	87	471.2	5	463.1	5	19	92	5.6
Dimacs Polblogs	1224	1224	33452	1072	1073	33123	51	6	303	153.8	3	83.8	5	77	382	4.9
Openflights	3409	3418	67685	2239	2238	35087	52	7	361	288.7	3	148.4	5	93	462	6.1
Appliances	1755732	104237	2128627	65120	9339	149046	3	3	6	4388.4	1	705.8	5	2	8	27.4
Beauty	631986	115709	701550	2968	2443	10542	4	4	13	252.0	2	100.6	5	3	13	2.5
Fashion	2035490	874297	2500961	5576	3861	15902	4	4	13	467.3	4	353.2	5	2	7	4.3
Music	100952	70519	130456	1078	910	2852	4	4	13	295.1	1	20.2	5	3	12	0.8
Edit Srwikiquote	402	1823	8694	133	759	3527	5	26	127	83.5	2	31.5	5	26	127	0.8
Edit Mrwiktionary	281	3995	14374	87	1724	6135	5	95	472	414.4	1	82.7	5	95	472	1.3
Edit Wowiktionary	193	3646	17799	57	1538	7761	5	354	1767	1271.0	1	441.0	5	354	1767	1.8

TABLE VII
$k = 4, \theta = 5$

Dataset	$\lvert U_G \rvert$	$\lvert V_G \rvert$	$\lvert E_G \rvert$	$\lvert U_R \rvert$	$\lvert V_R \rvert$	$\lvert E_R \rvert$	F_N						θ-Heuristic			
							$\lvert U_{D_{best}} \rvert$	$\lvert V_{D_{best}} \rvert$	$\lvert E(D_{best}) \rvert$	Discovery	ETB	TimTB	$\lvert U_{D_{best}} \rvert$	$\lvert V_{D_{best}} \rvert$	$\lvert E(D_{best}) \rvert$	Time
Maayan Vidal	1633	2929	6747	1633	2929	6747	5	5	21	199.1	1	39.1	5	4	16	1.6
P2P Gnutella09	3055	8038	26034	3055	8038	26034	5	5	21	536.6	3	284.8	5	2	6	3.9
Facebook	28	2885	3001	28	2885	3001	5	5	23	226.3	2	91.9	5	4	19	1.0
Edit Rowikibooks	443	4221	18547	443	4221	6994	5	6	26	379.1	2	164.8	5	5	21	2.0
Moreno Health	2313	2441	12990	2313	2441	12990	7	6	38	216.5	4	160.1	5	5	21	2.1
Matter	16726	22015	58616	16726	22015	58616	6	10	56	1566.4	4	1219.7	5	2	7	7.4
Gnutella	4935	10856	40015	4935	10856	40015	13	6	74	1232.2	4	972.0	5	2	6	11.2
Bitcoin	3286	3754	24207	3286	3754	24207	17	5	81	364.5	4	278.9	5	10	46	3.9
Edit Guwikiquote	963	1678	11385	963	1678	5523	5	33	161	271.0	5	265.1	5	20	96	1.4
Ciaodvd	1438	4299	40154	1438	4299	40154	34	5	166	609.6	3	336.9	5	15	71	7.9
Edit Nowikinews	430	4060	16921	430	4060	8912	5	35	171	527.7	2	205.1	5	11	51	2.3
Pgp Arenas	5596	9657	24337	5596	9657	24337	14	15	206	860.1	5	832.6	5	28	136	4.8
Moreno Blogs	1065	990	19046	1065	990	19046	54	6	320	155.1	5	149.0	5	6	26	2.4
Petster Hamster	2244	1766	16652	2244	1766	16652	70	5	346	210.9	2	73.3	5	21	101	2.4
Edit Mrwiktionary	281	3995	14373	281	3995	8519	5	96	476	441.5	2	180.7	5	3	11	1.9
Edit Vewiki	543	1532	11067	543	1532	5198	6	82	488	176.9	3	96.2	5	94	466	1.2
Dnc Corecipient	653	714	12106	653	714	12106	29	47	1359	163.0	5	154.1	5	91	451	1.8
Edit Jamwiki	177	2741	17667	177	2741	10409	5	298	1486	996.0	2	481.7	5	186	926	2.5
Nawiki	844	4014	70716	844	4014	25938	10	256	2556	746.5	1	126.4	5	433	2161	3.0
Edit Mkwikisource	218	6635	13822	218	6635	7749	4	4	12	206291.0	2	77831.0	5	5	21	2.5
Dblp Cite	3158	11566	49780	3158	11566	49780	5	19	91	585.4	5	577.0	5	20	96	4.5
Dimacs Polblogs	1224	1224	33451	1224	1224	33451	47	7	325	284.7	2	104.2	5	78	386	9.4
Openflights	3409	3418	67684	3409	3418	37616	55	7	381	762.7	4	555.1	5	94	466	10.5
Gift Cards	132732	1894	152431	132732	1894	151010	–	–	–	–	–	–	5	5	22	22.7
Beauty	631986	115709	701549	631986	115709	694273	3	3	5	82833.8	2	30546.7	5	3	11	128.6
Boxes	15237	1128	16237	15237	1128	16037	3	3	5	199761.7	1	2782.6	5	1	3	4.1
Health	461656	62597	494142	461656	62597	489011	3	3	5	49866.6	1	9855.9	5	4	16	89.7
Magazine	60144	3391	71518	60144	3391	70943	3	3	5	3202.6	1	583.8	5	3	11	10.0
Music	100952	70519	130455	100952	70519	128785	3	3	5	10403.2	1	1919.8	5	1	3	26.5
Moreno	888	1597	2298	888	1597	2298	4	5	16	120.9	2	49.3	5	1	2	0.7
Edit Srwikiquote	402	1823	8693	402	1823	4805	5	27	131	151.3	1	28.8	5	27	131	1.1
Edit Zuwiktionary	214	1404	7553	214	1404	4196	5	120	596	204.8	2	75.1	5	120	596	1.0
Edit Wowiktionary	193	3646	17798	193	3646	9941	5	355	1771	1202.1	1	358.3	5	355	1771	2.6
Appliances	1755732	104237	2128626	1755732	104237	2105969	–	–	–	–	–	–	5	2	8	370.0
Fashion	2035490	874297	2500960	2035490	874297	2475715	–	–	–	–	–	–	5	2	7	1212.0
Instruments	1762679	259791	3017460	1762679	259791	2983801	–	–	–	–	–	–	5	4	16	723.7
Wiki Talk Nds	605	23094	27453	605	23094	24689	–	–	–	–	–	–	5	3	12	5.5

TABLE VIII
$k = 3, \theta = 6$

Dataset	$\|U_G\|$	$\|V_G\|$	$\|E_G\|$	$\|U_R\|$	$\|V_R\|$	$\|E_R\|$	F_N						θ-Heuristic			
							$\|U_{D_{best}}\|$	$\|V_{D_{best}}\|$	$\|E(D_{best})\|$	Discovery	ETB	TimTB	$\|U_{D_{best}}\|$	$\|V_{D_{best}}\|$	$\|E(D_{best})\|$	Time
Appliances	1755732	104237	2128638	1290	422	4185	6	6	33	82.6	1	15.5	6	3	16	0.8
Beauty	631986	115709	701561	104	138	546	6	6	33	14.9	1	2.8	6	2	11	0.2
Fashion	2035490	874297	2500972	89	79	366	6	6	33	14.4	1	2.2	6	1	3	0.1
Health	461656	62597	494154	58	88	340	6	6	33	10.7	1	1.9	6	2	10	0.1
Maayan Vidal	1633	2929	6759	82	135	557	6	6	33	13.7	1	2.5	6	2	10	0.2
Magazine	60144	3391	71530	249	77	932	6	6	33	18.3	1	3.5	6	3	15	0.3
Music	100952	70519	130467	64	82	308	6	6	33	10.5	1	1.9	6	2	10	0.1
Moreno Health	2313	2441	13002	326	276	1506	7	6	39	30.1	2	11.6	6	6	34	0.4
Bitcoin	3286	3754	24219	1004	1105	15405	9	8	69	115.6	5	112.0	6	7	39	2.2
Edit Nowikinews	430	4060	16933	83	1193	5187	6	13	75	241.8	4	191.8	6	7	40	1.7
Gnutella	4935	10856	40027	422	228	2768	14	6	81	31.3	3	17.9	6	6	35	0.4
Matter	16726	22015	58628	2284	3277	11835	9	10	87	273.6	4	212.2	6	4	21	2.3
Ciaodvd	1438	4299	40166	978	1593	35095	10	10	97	149.0	3	81.3	6	12	69	3.7
Pgp Arenas	5596	9657	24349	937	1247	9801	15	14	207	112.4	4	86.3	6	25	148	1.6
Moreno Blogs	1065	990	19058	809	625	18055	16	17	269	111.6	3	62.0	6	3	16	1.9
Petster Hamster	2244	1766	16664	1311	852	13618	10	29	287	109.6	5	106.2	6	17	99	2.0
Dnc Corecipient	653	714	12116	393	369	11387	26	26	673	92.1	5	90.1	6	86	513	2.0
Edit Jamwiki	177	2741	17679	65	1485	8310	6	269	1611	738.5	3	538.9	6	75	447	1.9
Nawiki	844	4014	70728	205	1338	22043	12	196	2349	656.6	2	265.0	6	390	2337	5.1
Instruments	1762679	259791	3017472	6280	2154	29407	6	4	21	405.4	1	75.1	6	6	33	4.7
Dblp Cite	3158	11566	49792	2466	2956	31289	6	6	33	272.2	4	208.2	6	10	57	4.3
Edit Guwikiquote	963	1678	11397	130	649	3339	5	37	182	180.5	1	32.9	6	9	51	0.9
Dimacs Polblogs	1224	1224	33463	946	946	32537	17	15	252	138.1	4	105.7	6	59	351	5.0
Openflights	3409	3418	67696	1481	1485	31980	14	22	305	187.5	3	100.2	6	75	447	4.8
Edit Vewiki	543	1532	11079	105	275	3395	9	40	357	77.2	2	30.2	6	81	483	0.8
Gift Cards	132732	1894	152443	621	97	2270	4	4	13	45.8	1	7.9	6	4	21	0.5
Wiki Talk Nds	605	23094	27465	46	150	627	5	5	22	116.0	1	12.8	6	1	5	0.2
Edit Rowikibooks	443	4221	18559	70	426	1526	6	5	27	42.1	1	8.7	6	3	16	0.4
Edit Mkwikisource	218	6635	13834	20	124	416	7	5	32	24.5	5	23.9	6	1	4	0.2
Boxes	15237	1128	16249	15	15	60	6	6	33	5.4	1	0.9	6	6	33	0.0
Facebook	28	2885	3013	10	9	47	6	6	33	5.1	1	0.9	6	6	33	0.0
Moreno	888	1597	2310	14	13	60	6	6	33	5.2	1	0.9	6	6	33	0.0
P2P Gnutella09	3055	8038	26046	456	107	4066	6	6	33	29.5	2	11.5	6	6	33	0.5
Edit Srwikiquote	402	1823	8705	81	397	2647	6	13	75	52.7	4	41.4	6	13	75	0.6
Edit Mrwiktionary	281	3995	14385	47	1014	4641	6	52	309	312.3	2	157.7	6	52	309	1.2
Edit Zuwiktionary	214	1404	7565	37	559	3154	6	66	393	148.1	1	36.2	6	66	393	0.8
Edit Wowiktionary	193	3646	17810	26	1257	7185	6	196	1173	1113.5	1	422.0	6	196	1173	2.0

TABLE IX
$k = 3, \theta = 7$

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	$\lvert E_R\rvert$	F_N						θ-Heuristic			
							$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Discovery	ETB	TimTB	$\lvert U_{D_{best}}\rvert$	$\lvert V_{D_{best}}\rvert$	$\lvert E(D_{best})\rvert$	Time
Edit Mkwikisource	218	6635	13847	11	19	99	7	7	46	6.8	1	1.2	7	1	4	0.0
Edit Nowikinews	430	4060	16946	52	710	3538	7	7	46	135.8	1	27.6	7	4	25	0.8
Edit Rowikibooks	443	4221	18572	27	68	331	7	7	46	11.8	1	2.1	7	2	11	0.1
Instruments	1762679	259791	3017485	236	201	1503	7	7	46	25.4	1	4.6	7	6	40	0.4
Moreno Health	2313	2441	13015	41	30	221	7	7	46	8.9	1	1.5	7	5	33	0.1
P2P Gnutella09	3055	8038	26059	387	74	3483	7	7	46	25.4	1	4.8	7	4	25	0.6
Wiki Talk Nds	605	23094	27478	26	26	180	8	7	53	8.4	1	1.5	7	4	26	0.1
Gnutella	4935	10856	40040	256	141	1701	11	7	74	22.1	3	12.4	7	6	39	0.3
Bitcoin	3286	3754	24232	585	624	11234	12	7	81	73.0	2	26.4	7	10	68	1.6
Matter	16726	22015	58641	391	490	2316	8	12	93	44.7	1	8.4	7	8	53	0.5
Ciaodvd	1438	4299	40179	843	1158	32281	19	7	130	129.3	4	96.7	7	15	102	4.5
Petster Hamster	2244	1766	16677	968	629	11250	18	8	141	86.7	1	14.9	7	17	116	2.3
Edit Zuwiktionary	214	1404	7578	34	467	2870	7	30	207	129.4	2	54.0	7	25	172	0.8
Pgp Arenas	5596	9657	24362	576	743	7364	13	18	231	74.2	5	71.7	7	24	166	1.3
Moreno Blogs	1065	990	19070	705	501	17143	44	7	305	107.7	2	37.1	7	4	26	2.3
Dnc Corecipient	653	714	12127	343	326	11058	37	25	922	117.9	1	18.5	7	83	578	2.4
Edit Jamwiki	177	2741	17692	52	1167	7302	7	134	935	609.4	1	195.8	7	70	487	2.1
Nawiki	844	4014	70741	170	1120	21270	10	246	2457	754.1	4	570.2	7	346	2419	5.2
Dimacs Polblogs	1224	1224	33476	848	848	31744	7	44	305	131.7	4	95.9	7	48	333	5.0
Openflights	3409	3418	67708	1178	1178	30003	27	13	348	181.9	5	171.3	7	64	445	6.2
Edit Guwikiquote	963	1678	11410	88	403	2420	5	5	22	122.9	1	20.4	7	3	18	0.6
Dblp Cite	3158	11566	49805	1630	1455	18569	6	6	33	161.3	1	28.7	7	6	39	3.1
Appliances	1755732	104237	2128651	7	7	46	7	7	46	6.2	1	0.9	7	7	46	0.0
Beauty	631986	115709	701574	11	11	62	7	7	46	6.3	1	1.1	7	7	46	0.0
Boxes	15237	1128	16262	7	7	46	7	7	46	6.1	1	0.9	7	7	46	0.0
Edit Srwikiquote	402	1823	8718	53	293	2164	7	7	46	46.7	1	9.3	7	7	46	0.6
Facebook	28	2885	3026	8	7	51	7	7	46	6.0	1	1.0	7	7	46	0.0
Fashion	2035490	874297	2500985	7	7	46	7	7	46	6.6	1	0.9	7	7	46	0.0
Gift Cards	132732	1894	152456	30	22	142	7	7	46	7.4	1	1.3	7	7	46	0.0
Health	461656	62597	494167	7	7	46	7	7	46	6.4	1	1.0	7	7	46	0.0
Maayan Vidal	1633	2929	6772	15	15	78	7	7	46	6.5	1	1.0	7	7	46	0.0
Magazine	60144	3391	71543	11	11	62	7	7	46	6.6	1	1.0	7	7	46	0.0
Moreno	888	1597	2323	7	7	46	7	7	46	6.3	1	1.0	7	7	46	0.0
Music	100952	70519	130480	7	7	46	7	7	46	6.5	1	0.9	7	7	46	0.0
Edit Mrwiktionary	281	3995	14398	31	621	3405	7	18	123	215.7	5	213.6	7	18	123	0.9
Edit Vewiki	543	1532	11092	97	232	3240	7	60	417	82.6	2	29.2	7	60	417	1.0
Edit Wowiktionary	193	3646	17822	27	1018	6474	7	121	844	754.6	2	414.0	7	121	844	1.8

Table X lists per-graph results for the Tug-of-War (TOW) alternating best-response heuristic against ACO and the θ-heuristic. All three methods run on the same reduced graph; $|E_G|$ is the original edge count and $|U_R|$, $|V_R|$ are the reduced side sizes. $|U_D|$, $|V_D|$, and $|E(D)|$ denote the returned biclique, and Time is the wall time of one timed run after a JIT warmup run, in milliseconds.

TABLE X

TOW VS. ACO, θ-HEURISTIC ($k = 2, \theta = 5$, 37 GRAPHS). TOW VS. ACO: 16 WINS, 10 TIES, 11 LOSSES; VS. θ-HEURISTIC: 19 WINS, 6 TIES, 12 LOSSES. ROWS GROUPED BY WHETHER TOW BEATS EVERY OTHER METHOD, TIES FOR BEST, OR IS BEATEN. BOLD: LARGEST θ-FEASIBLE $|E(D)|$ PER ROW; †: NOT θ-FEASIBLE. TIMES IN MS. SETTINGS — ACO: 10 ANTS, 5 EPOCHS; TOW: MAX SHAKES=3, NUM SEEDS=10, PEEL=TRUE, SEED SOURCE=SC.

Dataset	$\lvert U_G\rvert$	$\lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert$	$\lvert V_R\rvert$	$\lvert E_R\rvert$	TOW				ACO				θ-Heuristic			
							$\lvert U_D\rvert$	$\lvert V_D\rvert$	$\lvert E(D)\rvert$	Time	$\lvert U_D\rvert$	$\lvert V_D\rvert$	$\lvert E(D)\rvert$	Time	$\lvert U_D\rvert$	$\lvert V_D\rvert$	$\lvert E(D)\rvert$	Time
Appliances	1755732	104237	2128628	1289	421	4175	5	5	**23**	9.8	5	4	18†	4.8	5	3	13†	0.6
Gift Cards	132732	1894	152433	620	96	2260	5	5	**23**	7.0	4	4	14†	2.7	5	4	18†	0.3
Health	461656	62597	494144	57	87	330	5	5	**23**	0.7	4	4	14†	0.7	5	2	8†	0.1
Maayan Vidal	–	–	6749	81	134	–	5	5	**23**	1.7	4	4	14†	1.4	5	3	13†	0.2
Music	100952	70519	130457	63	81	298	5	5	**23**	0.8	4	5	18†	0.9	5	3	13†	0.1
Edit Mkwikisource	–	–	13824	19	121	–	5	5	**24**	1.6	5	5	23	2.7	5	2	8†	0.1
Wiki Talk Nds	–	–	27455	45	149	–	6	5	**28**	3.0	4	4	14†	7.7	5	1	3†	0.2
Bitcoin	–	–	24209	1003	1105	–	14	5	**68**	362.2	5	12	58	11.2	5	8	38	2.4
Moreno Blogs	–	–	19048	809	625	–	64	5	**318**	774.3	23	10	228	11.5	5	3	13†	1.6
Dnc Corecipient	–	–	12108	392	369	–	38	38	**1443**	594.0	18	35	628	12.3	5	89	443	1.7
Edit Jamwiki	–	–	17669	64	1484	–	5	329	**1643**	552.8	6	218	1306	62.4	5	184	918	1.7
Beauty	631986	115709	701551	103	137	536	5	5	**23**	1.2	5	5	**23**	1.0	5	3	13†	0.1
Boxes	–	–	16239	14	14	–	5	5	**23**	0.3	5	5	**23**	0.6	5	5	**23**	0.0
Edit Rowikibooks	–	–	18549	69	425	–	5	5	**23**	10.6	4	4	14†	7.8	5	5	**23**	0.4
Facebook	–	–	3003	9	8	–	5	5	**23**	0.2	5	5	**23**	0.7	5	5	**23**	0.0
Fashion	2035490	874297	2500962	88	78	356	5	5	**23**	0.8	5	5	**23**	0.7	5	1	3†	0.1
Magazine	–	–	71520	248	76	–	5	5	**23**	3.8	5	5	**23**	1.8	5	3	14†	0.3
Moreno	–	–	2300	13	12	–	5	5	**23**	0.2	5	5	**23**	0.6	5	5	**23**	0.0
P2P Gnutella09	–	–	26036	455	106	–	5	5	**23**	40.0	4	5	18†	3.1	5	5	**23**	0.5
Petster Hamster	–	–	16654	1310	852	–	17	8	**135**	176.1	17	8	**135**	10.3	5	19	93	2.0
Edit Zuwiktionary	–	–	7555	36	559	–	5	118	**588**	74.7	5	118	**588**	16.6	5	49	243	0.7
Edit Wowiktionary	–	–	17800	25	1257	–	5	353	**1763**	418.5	5	353	**1763**	98.3	5	353	**1763**	1.8
Edit Nowikinews	–	–	16923	82	1192	–	5	5	23	108.8	4	45	178†	30.6	5	9	**43**	1.2
Edit Srwikiquote	–	–	8695	80	396	–	5	5	23	36.9	5	13	63	5.5	5	25	**123**	0.6
Instruments	1762679	259791	3017462	6279	2153	29397	5	5	23	186.8	4	4	14†	22.9	5	7	**33**	2.7
Moreno Health	–	–	12992	325	275	–	5	5	23	4.2	5	5	23	2.6	5	6	**29**	0.3
Gnutella	–	–	40017	421	227	–	6	6	34	12.6	7	7	**47**	3.2	5	6	30	0.5
Matter	–	–	58618	2283	3276	–	7	7	47	92.4	8	11	**87**	20.5	5	4	18†	2.0
Edit Guwikiquote	–	–	11387	129	649	–	5	10	48	44.3	5	14	68	23.2	5	18	**88**	0.9
Dblp Cite	–	–	49782	2462	2944	–	9	6	52	395.6	7	8	54	29.0	5	18	**88**	4.3
Ciaodvd	–	–	40156	978	1593	–	7	8	54	1883.9	16	6	**94**	14.1	5	13	63	3.3
Pgp Arenas	–	–	24339	936	1247	–	8	15	119	77.0	11	14	**152**	10.0	5	27	133	1.7
Dimacs Polblogs	–	–	33453	945	946	–	5	70	348	2399.1	64	5	318	17.3	5	76	**378**	4.4
Openflights	–	–	67686	1478	1485	–	8	47	374	1870.2	63	6	376	20.1	5	92	**458**	4.8
Edit Mrwiktionary	–	–	14375	46	1014	–	5	77	383	114.7	5	94	**468**	65.7	5	94	**468**	1.1
Edit Vewiki	–	–	11069	104	274	–	5	93	463	75.5	6	80	**478**	8.5	5	92	458	0.7
Nawiki	–	–	70718	204	1337	–	5	368	1838	3044.0	12	195	**2338**	78.5	5	431	2153	4.5

Table XI lists branch-and-pivot timings for every graph. $|U_G| + |V_G|$ and $|E_G|$ are the vertex and edge counts of the original graph; $|U_R|+|V_R|$ and $|E_R|$ are the corresponding counts after common-neighbor reduction.

Rows are grouped into two blocks separated by a horizontal rule: graphs where ACO seeding reduced net end-to-end time (top), and graphs with no net savings or overhead (bottom). Within each block, rows are sorted by ascending absolute time saved.

Time saved is defined as $T_{\text{pivot}}(\theta) - T_{\text{discovery}} - T_{\text{pivot}}(\text{ACO})$, where $T_{\text{discovery}}$ is the cost of running ACO for all 5 replicates. Time saved represents the net wall-clock time change of running branch-and-pivot with an ACO seed versus running branch-and-pivot on the θ-heuristic seed alone. Positive values mean the ACO-seeded pipeline is faster overall. Negative values mean ACO search did not beat the θ-heuristic, in which case time saved $= -T_{ACO}$. In some small graphs, ACO did beat the θ-heuristic, but the time it took to run ACO meant it did not result in overall time savings. Reduction (%) is defined as $100 \times$ Time saved$/T_{\text{pivot}}(\theta)$. Graphs where both ACO-seeded pivot and the θ-seeded pivot are omitted, as defined in Section 3.3.

Algorithm 4 Branch-and-Pivot

Require: A bipartite graph $G = (U, V, E)$, parameters k and θ
Ensure: A k-MDB of G, if one exists

```
D_best ← ThetaHeuristic(G)
BranchP(∅, (U, V))
return D_best
function BranchP(S = (U_S, V_S), C = (U_C, V_C))
    if C = ∅ then
        if |E(S)| > |E(D_best)| and |U_S| ≥ θ and |V_S| ≥ θ then
            D_best ← S
        return
    C_0 ← {v ∈ C | d̄_S(v) = 0}
    if C_0 = ∅ or |C \ C_0| ≥ k − |Ē(S)| then
        u ← argmax_{v∈C} d̄_S
        (S', C') ← Update(S, C, u)
        BranchP(S', C')
        BranchP(S, C \ {u})
    else
        u ← argmin_{v∈C_0} d̄_C
        if d̄_C(u) > k − |Ē(S)| > 0 then
            (S', C') ← Update(S, C, u)
            BranchP(S', C')
            BranchP(S, C \ {u})
        else
            for v ∈ {u} ∪ N_C(u) do
                (S', C') ← Update(S, C, v)
                BranchP(S', C')
                C ← C \ {v}
function Update(S, C, u)
    C' ← {v ∈ C \ {u} | d̄_{S∪{u}}(v) ≤ k − |Ē(S)|}
    C'_0 ← {v ∈ C' | d̄_{S∪{u}∪C'}(v) = 0}
    return (S ∪ {u} ∪ C'_0, C' \ C'_0)
```

TABLE XI
PIVOT SEED COMPARISON: ACO VS. θ-HEURISTIC (FULL RESULTS)

Dataset	$\lvert U_G\rvert + \lvert V_G\rvert$	$\lvert E_G\rvert$	$\lvert U_R\rvert + \lvert V_R\rvert$	$\lvert E_R\rvert$	Time saved (s)	Reduction (%)
Edit Jamwiki	2918	17669	1548	8298	6.26	6.52
Edit Zuwiktionary	1618	7555	595	3142	51.61	49.45
Edit Guwikiquote	2641	11387	778	3328	152.02	54.42
Edit Nowikinews	4490	16923	1274	5177	291.36	30.37
Edit Vewiki	2075	11069	378	3385	714.47	56.15
Music	171471	130457	144	298	-0.01	-0.00
Health	524253	494144	144	330	-0.01	-0.00
Fashion	2909787	2500962	166	356	-0.01	-0.00
Beauty	747695	701551	240	536	-0.01	-0.00
Magazine	63535	71520	324	922	-0.01	-0.00
Boxes	16365	16239	28	50	-0.03	-0.00
Facebook	2913	3003	17	36	-0.03	-0.00
Moreno	2485	2300	25	50	-0.03	-0.00
Maayan Vidal	4562	6749	215	547	-0.04	-26.51
Appliances	1859969	2128628	1710	4175	-0.04	-0.00
Gift Cards	134626	152433	716	2260	-0.06	-0.00
Moreno Health	4754	12992	600	1496	-0.07	-35.27
Gnutella	15791	40017	648	2758	-0.09	-7.67
Ciaodvd	5737	40156	2571	35082	-0.10	-0.01
Matter	38741	58618	5559	11825	-0.15	-0.01
P2P Gnutella09	11093	26036	561	4056	-0.17	-0.01
Edit Mkwikisource	6853	13824	140	399	-0.17	-0.01
Openflights	6827	67686	2965	31970	-0.24	-0.02
Wiki Talk Nds	23699	27455	194	616	-0.29	-25.39
Edit Rowikibooks	4664	18549	494	1515	-0.32	-0.02
Dblp Cite	14724	49782	5421	31279	-0.32	-0.03
Edit Srwikiquote	2225	8695	476	2637	-0.36	-0.03
Instruments	2022470	3017462	8432	29397	-0.44	-0.03
Nawiki	4858	70718	1541	22032	-0.50	-0.04
Dimacs Polblogs	2448	33453	1891	32524	-0.83	-0.07
Edit Mrwiktionary	4276	14375	1060	4630	-2.98	-0.23
Edit Wowiktionary	3839	17800	1282	7175	-8.45	-0.66